HTWK
Hochschule für Technik,
Wirtschaft und Kultur Leipzig

Fakultät Ingenieurwissenschaften

E203 Echtzeitsysteme und mobile Robotik

Echtzeitsysteme und mobile Robotik

Author Jonas Ledig, 86413
Mohamad Laith Kebab, 79765

Betreuer M.Sc. Moritz Thümmler
M.Sc. Nico Beyer

31. Oktober 2026

Inhaltsverzeichnis

1 Einleitung **1**

2 Teil 1: Praktikum Mobile Robotik **2**

2.1 Einführung in ROS 2 2

2.2 Erstellung eines eigenen ROS 2-Pakets 2

2.3 Programmierung eines eigenen Roboters 4

2.4 Erweiterung durch Sensorik und eigene Projekte 7

2.5 Arbeiten mit einem professionellen Roboter: TurtleBot4 11

2.6 Auswertung 17

3 Teil 2: Programmieraufgaben Echtzeitsysteme **18**

3.1 Sockets 18

3.1.1 Teilaufgabe a: Server 18

3.1.2 Teilaufgabe b: Client 19

3.1.3 Teilaufgabe c: Verwaltung einer gemeinsamen Liste 20

3.1.4 Teilaufgabe d: Peer-to-Peer-Erweiterung 20

3.2 Interprozesskommunikation (IPC) 21

3.2.1 Teilaufgabe a: Server-Prozess 21

3.2.2 Teilaufgabe b: Client-Prozess 22

3.2.3 Teilaufgabe c: Verwaltung einer gemeinsamen Liste 23

3.2.4 Teilaufgabe d: Gleichberechtigte Kommunikation 23

3.3 Auswertung 24

Appendix **25**

A shape_controller.py **25**

B 2.3 C++ Code **27**

C 2.4 C++ Code **32**

D obstacle_avoidance.py **39**

E wall_follower.py **43**

1 Einleitung

Im Rahmen des Moduls Echtzeitsysteme und mobile Robotik werden grundlegende Konzepte der Kommunikation, Steuerung und Programmierung verteilter und eingebetteter Systeme praktisch angewendet. Ein besonderer Schwerpunkt liegt dabei auf dem Robot Operating System 2 (ROS 2), welches eine entwicklungsfreundliche Kommunikation zwischen verschiedenen Softwarekomponenten ermöglicht.

Ziel des Praktikums ist es, die erlernten Kenntnisse durch die eigenständige Implementierung und Erprobung in verschiedenen Aufgaben zu vertiefen. Hierzu werden zunächst die Grundlagen von ROS 2 erarbeitet und mit Hilfe der Simulationsumgebung turtlesim erste Steuerungsprogramme entwickelt. Anschließend werden diese Kenntnisse auf einen mobilen Roboter mit ESP32-Mikrocontroller übertragen. Durch die Nutzung von micro-ROS wird dieser in das ROS-2-Netzwerk eingebunden, um Bewegungsbefehle zu empfangen und Sensordaten bereitzustellen. Abschließend erfolgt die Anwendung der erstellten Programme auf dem TurtleBot 4, der weiterführende Funktionen wie Kartierung und autonome Navigation besitzt.

Zusätzlich zur praktischen Arbeit mit den Robotersystemen werden verschiedene Programmieraufgaben zur Netzwerk- und Interprozesskommunikation bearbeitet. Dabei werden sowohl die Kommunikation über Sockets als auch der Datenaustausch zwischen lokalen Prozessen mithilfe von Message Queues untersucht. Die Aufgaben umfassen die schrittweise Entwicklung von Server-Client-Anwendungen, deren Erweiterung um eine gemeinsame Nachrichtenverwaltung sowie die Umsetzung einer gleichberechtigten Peer-to-Peer-Kommunikation.

Die vorliegende Dokumentation beschreibt die Durchführung der einzelnen Praktikumsaufgaben sowie die entwickelten Programme und deren Funktionsweise. Neben der technischen Umsetzung werden die Ergebnisse, aufgetretene Schwierigkeiten und die daraus gewonnenen Erkenntnisse dargestellt und ausgewertet.

Die verwendeten Quellcodes sind im GitHub-Repository unter https://github.com/LaithKu/EZMR-Beleg verfügbar.

2 Teil 1: Praktikum Mobile Robotik

2.1 Einführung in ROS 2

Nach der erfolgreichen Einrichtung von ROS 2 erfolgte zunächst eine Einarbeitung anhand der offiziellen ROS 2 Dokumentation. Hierbei wurden insbesondere die Struktur von ROS 2 sowie die Kommunikation zwischen einzelnen Nodes behandelt. Ein wesentlicher Bestandteil ist das Publisher-Subscriber-Prinzip, bei dem Nodes Nachrichten über sogenannte Topics austauschen.

Zur Veranschaulichung dieser Grundlagen wird in der offiziellen Dokumentation die Simulationsumgebung `turtlesim` vorgestellt. Mit ihrer Hilfe ist es möglich, eine virtuelle Schildkröte über ROS 2 Befehle zu bewegen und dabei die Funktionsweise von Nodes und Topics praktisch nachzuvollziehen. Zunächst kann die Schildkröte über eine Tastatursteuerung mit dem Befehl `ros2 run turtlesim_teleop_key` bewegt werden.

2.2 Erstellung eines eigenen ROS 2-Pakets

Aufbauend auf diesen Grundlagen bestand die Praktikumsaufgabe darin, ein eigenes ROS 2 Paket zu erstellen und die Bewegung der Schildkröte selbstständig zu programmieren. Anstelle einer manuellen Steuerung sollte die Schildkröte durch eine eigens entwickelte Node automatisch eine geometrische Form abfahren. Im Fall dieser Arbeit wurde sich für ein Quadrat entschieden.

Hierfür wurde im Workspace `~/ros2_ws` das Python-Paket `turtle_controller` mit der Node `shape_controller` erstellt.

Die Implementierung der Steuerungs Node erfolgte in Python unter Verwendung von `rclpy`, der Python-Clientbibliothek von ROS 2. Die erstellte Klasse `ShapeController` erbt von der Klasse `Node` aus dem Modul `rclpy.node`. Über den Aufruf des Konstruktors der Basisklasse wird die Node mit dem Namen `shape_controller` initialisiert.

Innerhalb der Node wird ein Publisher erstellt, der Nachrichten des Typs `geometry_msgs.msg/Twist` auf dem Topic `/cmd_vel` veröffentlicht. Der Nachrichtentyp enthält jeweils einen dreidimensionalen Vektor für die lineare und die angulare Geschwindigkeit. Für die Bewegung der Schildkröte werden ausschließlich die Komponenten `linear.x` und `angular.z` verwendet.

```
6  class ShapeController(Node):
7
8      def __init__(self):
9          super().__init__('shape_controller')
10
11         self.publisher_ = self.create_publisher(
12             Twist,
13             '/cmd_vel',
14             10
15         )
```

Quellcode 1: Initialisierung der ROS 2 Node (Anhang A)

Die Steuerung erfolgt über einen periodischen Timer mit einem konfigurierten Intervall von 0,1 Sekunden. Bei jedem Timerereignis wird die Methode `timer_callback()` ausgeführt, welche abhängig vom aktuellen Bewegungszustand eine neue Geschwindigkeitsnachricht erzeugt und veröffentlicht.

```
17         self.timer_period = 0.1
18         self.timer = self.create_timer(
19             self.timer_period,
20             self.timer_callback
21         )
```

Quellcode 2: Timer_Callback der ROS 2 Node (Anhang A)

Die Bewegungslogik ist als einfache Zustandsmaschine mit zwei Zuständen implementiert. Im ersten Zustand `self.state = 0` wird eine Geradeausfahrt mit einer linearen Sollgeschwindigkeit von 1,0 Einheiten pro Sekunde vorgegeben. Nach 20 Timeraufrufen erfolgt der Wechsel in den zweiten Zustand `self.state = 1`, in dem eine Drehung mit einer Winkelgeschwindigkeit von 1,57 rad/s ausgeführt wird. Nach weiteren zehn Timeraufrufen wird wieder in den ersten Zustand gewechselt und der Seitenzähler erhöht.

Nach vier abgeschlossenen Bewegungszyklen werden die Geschwindigkeitskomponenten auf null gesetzt. Dadurch wird die Bewegung beendet (`mgs.linear.x = 0.0 ; mgs.angular.z = 0.0`), während die Node weiterhin aktiv bleibt und Stoppbefehle veröffentlicht.

```
24          self.state = 0
25
26          self.counter = 0
27
28          self.side = 0
29
30          self.get_logger().info('Shape Controller gestartet')
31
32
33      def timer_callback(self):
34
35          msg = Twist()
36
37
38          if self.state == 0:
39
40              msg.linear.x = 1.0
41              msg.angular.z = 0.0
42
43              self.counter += 1
44
45              if self.counter >= 20:
46
47                  self.counter = 0
48                  self.state = 1
49
50          elif self.state == 1:
51
52              msg.linear.x = 0.0
53              msg.angular.z = 1.57
54
55              self.counter += 1
56
57              if self.counter >= 10:
58
59                  self.counter = 0
60                  self.state = 0
61
62                  self.side += 1
63
64                  self.get_logger().info(
65                      f'Seite {self.side} fertig'
66                  )
67
68          if self.side >= 4:
69
70              msg.linear.x = 0.0
71              msg.angular.z = 0.0
72
73          self.publisher_.publish(msg)
```

Quellcode 3: Zustandsmaschine der ROS 2 Node (Anhang A)



Abbildung 1: Shapecontroller in TurtleSim

Nach Ausführen des Shapecontrollers ist wie in Abbildung 1 zu sehen, die Turtle einmal im Viereck gefahren und nach allen Bewegungen wieder gestoppt, bis ein neuer Befehl ausgeführt wird.

Durch die Verwendung einer Zustandsmaschine ist es möglich eine übersichtliche Trennung der einzelnen Bewegungsabläufe zu gewährleisten. Allerdings erfolgt die Steuerung ausschließlich zeitbasiert. Die zurückgelegte Strecke und der Drehwinkel werden nicht anhand einer Positions- oder Orientierungsmessung überprüft, sondern aus den vorgegebenen Geschwindigkeiten und Zeitintervallen bestimmt. Dadurch entstanden Abweichungen von der theoretisch vorgesehenen quadratischen Bahn. Diese sind leicht auf der Abbildung zu erkennen, da die beiden vertikalen Spuren nicht exakt vertikal verlaufen.

2.3 Programmierung eines eigenen Roboters

Der nächste Teil der Praktikumsaufgabe besteht darin, einen realen Roboter mithilfe von ROS 2 anzusteuern. Hierbei sollte zunächst die zuvor entwickelte Steuerungs Node unverändert weiterverwendet werden. Dazu musste der Roboter in der Lage sein, die über das Topic `/cmd_vel` veröffentlichten Bewegungsbefehle zu empfangen und in entsprechende Motorbewegungen umzusetzen.

Der verwendete Roboter verfügt über einen ESP32 Mikrocontroller sowie zwei Schrittmotoren des Typs 28BYJ-48. Die Ansteuerung der Motoren erfolgt über jeweils einen ATtiny212 Mikrocontroller, der die erforderlichen Schrittfolgen erzeugt. Der ESP32 übernimmt dabei die Kommunikation mit ROS 2 und die Weiterleitung der Bewegungsbefehle an die Motorsteuerungen.

Damit sich der Mikrocontroller in ein bestehendes ROS 2 System einbinden kann, wurde die Bibliothek `micro-ROS` verwendet. Die Programmierung des ESP32 erfolgte mit Visual Studio Code

und PlatformIO unter Verwendung des Arduino Frameworks. Die Kommunikation zwischen dem ESP32 und dem Ubuntu System wurde über WLAN realisiert. Hierzu wurde auf Ubuntu ein micro-ROS-Agent gestartet, welcher als Vermittlungsinstanz zwischen dem Mikrocontroller und dem ROS 2 Netzwerk dient. Die Übertragung erfolgte über UDP auf Port 8888. Im Rahmen der Einrichtung wurde auf beiden Systemen die `ROS_DOMAIN_ID = 1` verwendet, damit die beteiligten Nodes derselben ROS 2 Domäne angehören. Die Netzwerkverbindung zwischen dem ESP32 und dem Ubuntu-System wurde über WLAN hergestellt. Zusätzlich wurde auf Ubuntu sichergestellt, dass die ROS 2 Kommunikation nicht ausschließlich auf die lokale Loopback Schnittstelle beschränkt war.

Auf dem ESP32 wurde ein micro-ROS Subscriber eingerichtet, welcher Nachrichten des Typs geometry_msgs/msg/Twist über das Topic /cmd_vel empfängt. Die Initialisierung erfolgt innerhalb der Funktion setup(), in der der Subscriber auch beim micro-ROS Executor registriert wird. Bei Eingang einer neuen Nachricht wird die Funktion subscription_callback() aufgerufen, welche die empfangenen Geschwindigkeitskomponenten auswertet und in entsprechende Fahrbefehle für die beiden Schrittmotoren umsetzt.

```
277     RCCHECK(
278         rclc_subscription_init_default(
279             &subscriber,
280             &node,
281             ROSIDL_GET_MSG_TYPE_SUPPORT(
282                 geometry_msgs,
283                 msg,
284                 Twist
285             ),
286             "/cmd_vel"
287         )
288     );
289
290     RCCHECK(
291         rclc_executor_init(
292             &executor,
293             &support.context,
294             1,
295             &allocator
296         )
297     );
298
299     RCCHECK(
300         rclc_executor_add_subscription(
301             &executor,
302             &subscriber,
303             &msg,
304             &subscription_callback,
305             ON_NEW_DATA
306         )
307     );
```

Quellcode 4: Initialisierung des micro-ROS-Subscribers (Anhang B)

Bei der Inbetriebnahme des Roboters traten zunächst Schwierigkeiten bei der Ansteuerung der beiden Schrittmotoren auf. Die Motoren werden jeweils über einen ATtiny212-Mikrocontroller angesteuert, welcher die Befehle über eine UART-Schnittstelle vom ESP32 empfängt. Zunächst war jedoch unklar, mit welchen Kommunikationseinstellungen und Befehlsparametern die Motorsteuerungen zuverlässig angesprochen werden konnten.

Zur Untersuchung wurde ein Diagnoseprogramm verwendet, mit dem verschiedene Baudraten, Motor IDs und Byte Reihenfolgen der übertragenen Parameter getestet wurden. Bei einer Baudrate von 9600 Baud wurde erstmals eine Bewegung der Motoren beobachtet. Durch weitere Tests wurde festgestellt, dass die Motor ID 1 dem linken und die Motor ID 2 dem rechten Motor zugeordnet ist.

Anschließend wurden die verschiedenen Betriebsmodi und Parameter der Motorsteuerung untersucht. Dabei konnte unter anderem mit dem Betriebsmodus 0 (Schrittmodus) eine Motorbewegung

erzeugt werden. Für die spätere Ansteuerung wurde der Betriebsmodus 1 (Dauerbetrieb) verwendet, welcher eine kontinuierliche Bewegung ermöglicht.

Ein weiteres Problem stellte die Wahl der Schrittgeschwindigkeit dar. Der 16-Bit Wert von `param` legte die Schrittfrequenz fest. Dabei verhielt sich der Wert umgekehrt zur Drehgeschwindigkeit. Ein kleiner Wert stand für eine hohe Frequenz und eine schnelle Drehung. Jedoch erwies sich ein zu kleiner Wert als unbrauchbar, da die Reibung und das Drehmoment die Motoren überlasteten und sich der Roboter nicht bewegte. Durch weiteres Austesten wurde der Grenzwert von 3 gefunden. Bei diesem Parameter ist eine flüssige und zuverlässige Inbetriebnahme möglich. Zusätzlich mussten die Drehrichtungen der beiden Motoren aufeinander abgestimmt werden. Da diese spiegelbildlich montiert sind, benötigen sie für eine gemeinsame Vorwärtsbewegung unterschiedliche Drehrichtungsparameter. Für den linken Motor wurde hierfür die Drehrichtung 1 und für den rechten Motor die Drehrichtung 0 verwendet.

Die ermittelten Einstellungen wurden in der Funktion sendMotorCommand() zusammengefasst. Diese überträgt einen sechs Byte langen Steuerbefehl über die UART-Schnittstelle an die Motorsteuerungen. Der Befehl besteht aus einem Startbyte, der Motor ID, dem Betriebsmodus, der Drehrichtung und dem 16-Bit-Parameter. Letzterer wird in zwei Bytes aufgeteilt, wobei zunächst das höherwertige und anschließend das niederwertige Byte übertragen wird.

```
32 void sendMotorCommand(
33     uint8_t id,
34     uint8_t mode,
35     uint8_t dir,
36     uint16_t param)
37 {
38     MotorSerial.write(START_BYTE);
39     MotorSerial.write(id);
40     MotorSerial.write(mode);
41     MotorSerial.write(dir);
42 
43     MotorSerial.write((uint8_t)((param >> 8) & 0xFF));
44     MotorSerial.write((uint8_t)(param & 0xFF));
45 
46     MotorSerial.flush();
47 }
```

Quellcode 5: Übertragung der UART Steuerbefehle (Anhang B)

Auf Grundlage der ermittelten Motorparameter wurden die Fahrfunktionen für Vorwärts- und Rückwärtsbewegungen sowie Links- und Rechtsdrehungen implementiert.

```
154 void subscription_callback(const void *msgin)
155 {
156     const geometry_msgs__msg__Twist *cmd =
157         (const geometry_msgs__msg__Twist *)msgin;
158 
159     float linear  = cmd->linear.x;
160     float angular = cmd->angular.z;
161 
162     Serial.print("linear.x = ");
163     Serial.print(linear);
164 
165     Serial.print(" | angular.z = ");
166     Serial.println(angular);
167 
168     // Vorwaerts
169     if (linear > 0.05 && abs(angular) < 0.05)
170     {
171         Serial.println("VORWAERTS");
172         driveForward();
173     }
174 
175     // Rueckwaerts
176     else if (linear < -0.05 && abs(angular) < 0.05)
177     {
```

```
178         Serial.println("RUECKWAERTS");
179         driveBackward();
180     }
181
182     // Links
183     else if (angular > 0.05)
184     {
185         Serial.println("LINKS");
186         turnLeft();
187     }
188
189     // Rechts
190     else if (angular < -0.05)
191     {
192         Serial.println("RECHTS");
193         turnRight();
194     }
```

Quellcode 6: Auswertung der ROS 2 Bewegungsbefehle (Anhang B)

Die Auswahl der jeweiligen Fahrfunktion erfolgt innerhalb der Funktion subscription_callback(). Hierbei werden die empfangenen Komponenten linear.x und angular.z der Twist-Nachricht ausgewertet. Anhand ihrer Vorzeichen und eines Schwellwerts von 0,05 wird die gewünschte Bewegungsrichtung bestimmt. Eine direkte Umrechnung der Geschwindigkeitswerte in physikalische Fahrgeschwindigkeiten erfolgt nicht, da die tatsächliche Schrittgeschwindigkeit durch den festgelegten Motorparameter vorgegeben wird.

Nach der Implementierung wurde zunächst die grundlegende Kommunikation zwischen dem ESP32 und dem ROS 2 System überprüft. Danach erfolgte die Erprobung der einzelnen Bewegungsrichtungen durch das Veröffentlichen entsprechender Nachrichten auf dem Topic `/cmd_vel`. Zur manuellen Überprüfung wurde außerdem die ROS 2 Node `teleop_twist_keyboard` wie bei der Turtlesim verwendet, um den Roboter mit der Tastatur zu steuern. Nach erfolgreicher Inbetriebnahme konnte auch die zuvor entwickelte Node `shape_controller` verwendet werden, um die Bewegungsabläufe automatisch auszuführen. Eine Anpassung der Python-Steuerungslogik war hierfür nicht erforderlich, da sowohl die Simulation als auch der reale Roboter das Topic `/cmd_vel` verwenden. Bei der Ausführung des Shapecontrollers auf dem realen Roboter zeigten sich deutliche Abweichungen von der vorgesehenen quadratischen Bahn. Da die Bewegungssteuerung ausschließlich zeitbasiert erfolgt, werden die tatsächlichen Fahrstrecken und Drehwinkel nicht berücksichtigt. Insbesondere erreichte der reale Roboter innerhalb der vorgegebenen Drehdauer keinen Winkel von 90°, da seine tatsächliche Drehgeschwindigkeit von der in turtlesim simulierten Bewegung abwich. Da die Steuerungs Node keine Rückmeldung über die Position oder Orientierung erhält, können diese Abweichungen nicht automatisch korrigiert werden. Folglich konnte der Roboter die vorgesehene quadratische Bahn nicht exakt abfahren.

2.4 Erweiterung durch Sensorik und eigene Projekte

Im nächsten Teil des Praktikums wurde der zuvor programmierte Roboter um zusätzliche Sensorik erweitert. Ziel war es, eine autonome Hindernisvermeidung zu implementieren, bei der der Roboter Hindernisse selbstständig erkennt und seine Fahrtrichtung entsprechend anpasst. Hierfür wurden ein Ultraschallsensor des Typs HC-SR04 zur Abstandsmessung sowie ein Gyroskop zur Erfassung der Drehbewegung verwendet. Die bereits implementierte Motorsteuerung aus Aufgabe 2.3 konnte dabei weiterverwendet werden.

Der Ultraschallsensor wurde über die GPIO-Pins 26 (Trigger) und 25 (Echo) mit dem ESP32 verbunden. Zur Abstandsmessung erzeugt der Mikrocontroller einen Triggerimpuls von 10 µs. Anschließend wird die Dauer des zurückkehrenden Echos gemessen. Aus der gemessenen Laufzeit und der Schallgeschwindigkeit lässt sich die Entfernung zum Hindernis berechnen. Da der Schall den Weg zum Hindernis und zurück zurücklegt, muss die gemessene Laufzeit halbiert werden.

Die Berechnung erfolgt nach folgender Gleichung:

$$s = \frac{c \cdot t}{2}$$

Dabei bezeichnet s die Entfernung zum Hindernis, c die Schallgeschwindigkeit und t die gemessene Laufzeit des Ultraschallsignals. Im Programm wurde eine Schallgeschwindigkeit von 343 m/s angenommen. Die Funktion ‘measureDistance()‘ übernimmt die Erzeugung des Triggerimpulses, die Messung der Echozeit sowie die Umrechnung der Entfernung in Meter. Wird innerhalb von 30 ms kein Echo empfangen, liefert die Funktion den Wert -1 zurück.

```
218 float measureDistance()
219 {
220     digitalWrite(TRIG_PIN, LOW);
221     delayMicroseconds(2);
222 
223     digitalWrite(TRIG_PIN, HIGH);
224     delayMicroseconds(10);
225 
226     digitalWrite(TRIG_PIN, LOW);
227 
228     unsigned long duration =
229         pulseIn(ECHO_PIN, HIGH, 30000);
230 
231     if (duration == 0)
232     {
233         return -1.0;
234     }
235 
236     float distance_cm =
237         duration * 0.0343 / 2.0;
238 
239     return distance_cm / 100.0;
240 }
```

Quellcode 7: Abstandsmessung mit dem HC-SR04 (Anhang C)

Zur Erfassung der Drehbewegung wurde zusätzlich ein Gyroskop über die I²C-Schnittstelle des ESP32 angebunden. Hierfür wurden die GPIO-Pins 21 (SDA) und 22 (SCL) verwendet. Das Gyroskop wird über die I²C-Adresse 0x68 angesprochen. Während der Initialisierung wird der Sensor aus dem Schlafmodus versetzt und ein Messbereich von ±500 °/s eingestellt.

Da Gyroskope auch im Stillstand eine geringe Winkelgeschwindigkeit messen können, wurde eine Offset-Kalibrierung implementiert. Hierzu werden beim Programmstart 200 Messwerte aufgenommen, während sich der Roboter nicht bewegt. Aus diesen Messwerten wird der Mittelwert berechnet und als Offset gespeichert. Bei den anschließenden Messungen wird dieser Offset von der erfassten Winkelgeschwindigkeit abgezogen, um systematische Messabweichungen zu reduzieren.

```
280 void calibrateGyro()
281 {
282     const int samples = 200;
283 
284     float sum = 0.0;
285 
286     Serial.println();
287     Serial.println("Gyro-Kalibrierung");
288     Serial.println("Roboter NICHT bewegen!");
289 
290     delay(1000);
291 
292     for (int i = 0; i < samples; i++)
293     {
294         sum += readGyroZ();
295         delay(10);
296     }
297 
```

```
    gyroBiasZ = sum / samples;

    Serial.print("Gyro-Bias Z = ");
    Serial.print(gyroBiasZ, 3);
    Serial.println(" deg/s");

    Serial.println(
        "Gyro-Kalibrierung abgeschlossen");
}
```

Quellcode 8: Kalibrierung des Gyroskops (Anhang C)

Die Messwerte beider Sensoren werden vom ESP32 über micro-ROS an das Ubuntu System übertragen. Hierfür wurden zwei Publisher eingerichtet, welche Nachrichten des Typs ‘std_msgs/msg/Float32‘ veröffentlichen. Die gemessene Entfernung wird über das Topic ‘/distance‘ in Metern übertragen. Die korrigierte Winkelgeschwindigkeit um die Z-Achse wird über das Topic ‘/gyro_z‘ in Grad pro Sekunde veröffentlicht. Die Veröffentlichung erfolgt zyklisch in einem Intervall von ungefähr 100 ms.

```
void loop()
{
    RCSOFTCHECK(
        rclc_executor_spin_some(
            &executor,
            RCL_MS_TO_NS(10)));

    static unsigned long lastSensorTime = 0;

    unsigned long now = millis();

    if (now - lastSensorTime >= 100)
    {
        lastSensorTime = now;

        float distance = measureDistance();

        distance_msg.data = distance;

        RCSOFTCHECK(
            rcl_publish(
                &distance_publisher,
                &distance_msg,
                NULL));

        float gyroZ =
            readGyroZ() - gyroBiasZ;

        if (fabs(gyroZ) < 1.0)
        {
            gyroZ = 0.0;
        }

        gyro_msg.data = gyroZ;

        RCSOFTCHECK(
            rcl_publish(
                &gyro_publisher,
                &gyro_msg,
                NULL));
```

Quellcode 9: Zyklische Veröffentlichung der Sensordaten (Anhang C)

Die Auswertung der Sensordaten und die autonome Bewegungssteuerung erfolgen auf dem Ubuntu System. Hierzu wurde eine Python-Node mit dem Namen ‘obstacle_avoidance‘ entwickelt. Diese abonniert die beiden Sensortopics ‘/distance‘ und ‘/gyro_z‘ und veröffentlicht die berechneten Bewegungsbefehle über ‘/cmd_vel‘. Der ESP32 empfängt diese Nachrichten und setzt sie mithilfe der bereits vorhandenen Motorfunktionen in Fahrbewegungen um. Dadurch war keine grundlegende Änderung der Motorsteuerung aus Aufgabe 2.3 erforderlich.

Zur Steuerung des Roboters wurde eine Zustandsmaschine implementiert, die in einem Intervall von 50 ms ausgeführt wird. Im Ausgangszustand ‘FORWARD‘ fährt der Roboter geradeaus. Sobald der Ultraschallsensor ein Hindernis in einer Entfernung von weniger als 0,25 m erkennt, wechselt die Steuerung in den Zustand ‘BACKWARD‘. In diesem Zustand fährt der Roboter für eine Sekunde rückwärts. Anschließend hält er für 0,3 Sekunden an, bevor eine Linksdrehung eingeleitet wird.

Im Zustand ‘TURN‘ wird der Roboter so lange gedreht, bis ein Winkel von mindestens 90° erreicht wurde. Hierfür wird die vom Gyroskop gemessene Winkelgeschwindigkeit über die Zeit integriert. Die Berechnung erfolgt durch Multiplikation der Winkelgeschwindigkeit mit dem Zeitintervall zwischen zwei aufeinanderfolgenden Messungen. Die einzelnen Winkeländerungen werden anschließend aufsummiert, sodass sich der gesamte zurückgelegte Drehwinkel ergibt.

```
70      def gyro_callback(self, msg):
71
72          self.gyro_z = msg.data
73
74          if self.state != 'TURN':
75              self.last_gyro_time = None
76              return
77
78          now = self.get_clock().now().nanoseconds / 1e9
79
80          if self.last_gyro_time is None:
81              self.last_gyro_time = now
82              return
83
84          dt = now - self.last_gyro_time
85          self.last_gyro_time = now
86
87          self.angle += self.gyro_z * dt
```

Quellcode 10: Berechnung des Drehwinkels aus den Gyroskopdaten (Anhang D)

Nach Erreichen des vorgesehenen Drehwinkels wird die Drehbewegung beendet. Der Roboter hält erneut für 0,3 Sekunden an und wechselt anschließend zurück in den Zustand ‘FORWARD‘. Auf diese Weise wird die Hindernisvermeidung wiederholt, sobald ein weiteres Hindernis erkannt wird.

```
137     def control_loop(self):
138
139         now = self.get_clock().now().nanoseconds / 1e9
140
141         elapsed = now - self.state_start_time
142
143         if self.state == 'FORWARD':
144
145             self.publish_forward()
146
147             if (
148                 self.distance > 0.0
149                 and self.distance < self.DISTANCE_THRESHOLD
150             ):
151
152                 self.get_logger().info(
153                     f'Hindernis erkannt: '
154                     f'{self.distance:.3f} m'
155                 )
156
157                 self.change_state('BACKWARD')
```

Quellcode 11: Hinderniserkennung und Einleitung der Ausweichbewegung (Anhang D)

Im Gegensatz zur zuvor implementierten zeitbasierten Steuerung wird die Drehbewegung nun anhand der gemessenen Winkelgeschwindigkeit überwacht. Dadurch kann der Drehvorgang unabhängig von einer fest vorgegebenen Drehdauer beendet werden. Allerdings können Messrauschen, verbleibende Offsetfehler und die zeitliche Integration der Winkelgeschwindigkeit zu Abweichungen beim ermittelten Drehwinkel führen. Auch die Hinderniserkennung ist von der Zuverlässigkeit der

Ultraschallmessung abhängig. Bei einem fehlenden Echo wird der Wert -1 übertragen, welcher von der Python Node nicht als gültige Hindernismessung berücksichtigt wird.

Nach Implementierung wurde die Obstacle Avoidance praktisch getestet. Dabei konnte die vorgesehene Bewegungsabfolge erfolgreich umgesetzt werden. Der Roboter bewegte sich zunächst vorwärts, bis der Ultraschallsensor ein Hindernis erkannte. Anschließend stoppte er, fuhr rückwärts, führte eine Linksdrehung aus und setzte seine Vorwärtsfahrt fort. Damit konnte die grundlegende Funktionsfähigkeit der entwickelten Hindernisvermeidung bestätigt werden.

Während der Rückwärtsfahrt wurde jedoch eine Abweichung vom vorgesehenen geradlinigen Bewegungsverlauf beobachtet. Das linke Rad drehte sich rückwärts langsamer als das rechte, wodurch der Roboter leicht nach rechts abdriftete. Da beide Motoren mit demselben Geschwindigkeitsparameter angesteuert wurden, deutet die Beobachtung auf Unterschiede im tatsächlichen Laufverhalten der beiden Antriebe hin. Eine mögliche Verbesserung wäre, die bereits vorhandene Gyroskopmessung auch während der Rückwärtsfahrt zur Erfassung dieser unbeabsichtigten Drehung zu verwenden. Die gemessene Winkelabweichung könnte anschließend bei der vorgesehenen 90°-Linksdrehung berücksichtigt werden, sodass die ursprüngliche Orientierung genauer eingehalten wird.

2.5 Arbeiten mit einem professionellen Roboter: TurtleBot4

Im letzten Teil des Praktikums zur mobilen Robotik wurde der TurtleBot4 eingesetzt. Im Gegensatz zum zuvor programmierten ESP32 Roboter verfügt dieser über eine vorkonfigurierte ROS 2 Umgebung sowie zusätzliche Funktionen zur Lokalisierung, Kartierung und autonomen Navigation. Ziel war es, die bisher erworbenen Kenntnisse auf einen professionellen mobilen Roboter zu übertragen und die erweiterten Möglichkeiten seiner Sensorik und Software zu untersuchen.

Zunächst wurde die grundlegende Kommunikation mit dem TurtleBot4 betrachtet. Nach dem Start verbindet sich der Roboter mit dem Netzwerk und stellt verschiedene ROS 2 Nodes und Topics bereit. Dadurch können sowohl Bewegungsbefehle übertragen als auch Sensordaten ausgelesen werden, ohne dass hierfür eine eigene Mikrocontroller-Firmware entwickelt werden muss. Im ersten Schritt wurde der TurtleBot4 mithilfe der Teleop Steuerung manuell durch den Raum bewegt. Dadurch konnten die grundlegenden Fahrfunktionen erstmals am realen Roboter getestet und dessen Bewegungsverhalten untersucht werden. Dabei fiel auf, dass die Rückwärtsfahrt nur eingeschränkt möglich war. Nach einer zurückgelegten Strecke von ungefähr 10 cm stoppte der Roboter und ließ anschließend lediglich Vorwärtsfahrten und Drehbewegungen zu. Gleichzeitig leuchtete der LED Ring gelb auf. Die Ursache hierfür liegt sehr wahrscheinlich in der integrierten Sicherheitsfunktion „Backup Limit“ der verwendeten iRobot Create 3 Plattform. Da sich die vier Absturzsensoren ausschließlich in der vorderen Hälfte des Roboters befinden, können mögliche Abgründe während der Rückwärtsfahrt nicht zuverlässig erkannt werden. Aus diesem Grund begrenzt die Sicherheitsfunktion die Rückwärtsbewegung auf zuvor als sicher erfasste Bereiche. Wird diese Grenze erreicht, stoppt der Roboter und signalisiert die Einschränkung über seinen LED Ring. Erst nach einer ausreichenden Vorwärtsbewegung wird die Rückwärtsfahrt wieder freigegeben.

Im nächsten Schritt wurden die vom TurtleBot4 bereitgestellten Sensordaten mithilfe von RViz und der ROS 2 Kommandozeile untersucht. RViz ermöglicht die grafische Darstellung der Roboterposition sowie der erfassten Umgebung. Hierfür werden unter anderem die Messwerte des integrierten 2D LiDAR Scanners verwendet. Parallel dazu wurde der Roboter über die Teleop Steuerung bewegt, um die Darstellung der Umgebung während der Fahrt zu untersuchen.

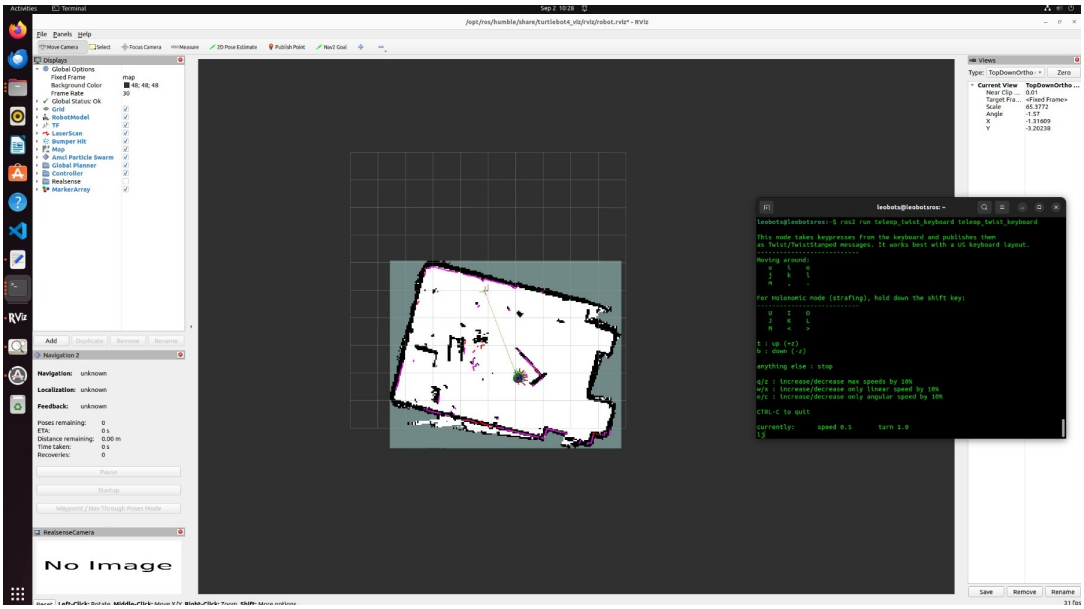


Abbildung 2: Visualisierung der Umgebung in RViz und manuelle Steuerung des TurtleBot4 mittels Teleop

Die in Abbildung 2 dargestellte RViz Ansicht zeigt eine zweidimensionale Karte des Raumes. Diese basiert auf dem SLAM Verfahren (Simultaneous Localization and Mapping), bei dem während der Bewegung des Roboters gleichzeitig dessen Position geschätzt und eine Karte der Umgebung erstellt wird. Dabei kennzeichnen weiße Bereiche die erfassten freien Flächen, schwarze Bereiche Hindernisse beziehungsweise Begrenzungen und graue Bereiche bislang unbekannte Flächen. Durch die fortlaufende Auswertung der LiDAR Messwerte kann die Karte schrittweise erweitert werden.

Ergänzend wurden Sensordaten in einem separaten Fenster wie in Abbildung 3 ausgegeben. Die dort dargestellten numerischen Abstandswerte ermöglichen es, die vom Roboter erfassten Entfernungen unabhängig von der grafischen Darstellung zu betrachten. Somit konnten sowohl die räumliche Zuordnung der Messwerte in RViz als auch deren numerische Ausgabe untersucht werden. Die auf diese Weise erstellte Karte kann gespeichert und anschließend für die autonome Navigation verwendet werden. Damit bildet die Kartierung die Grundlage für den nächsten Arbeitsschritt, bei dem der TurtleBot4 selbstständig eine vorgegebene Zielposition anfahren soll.

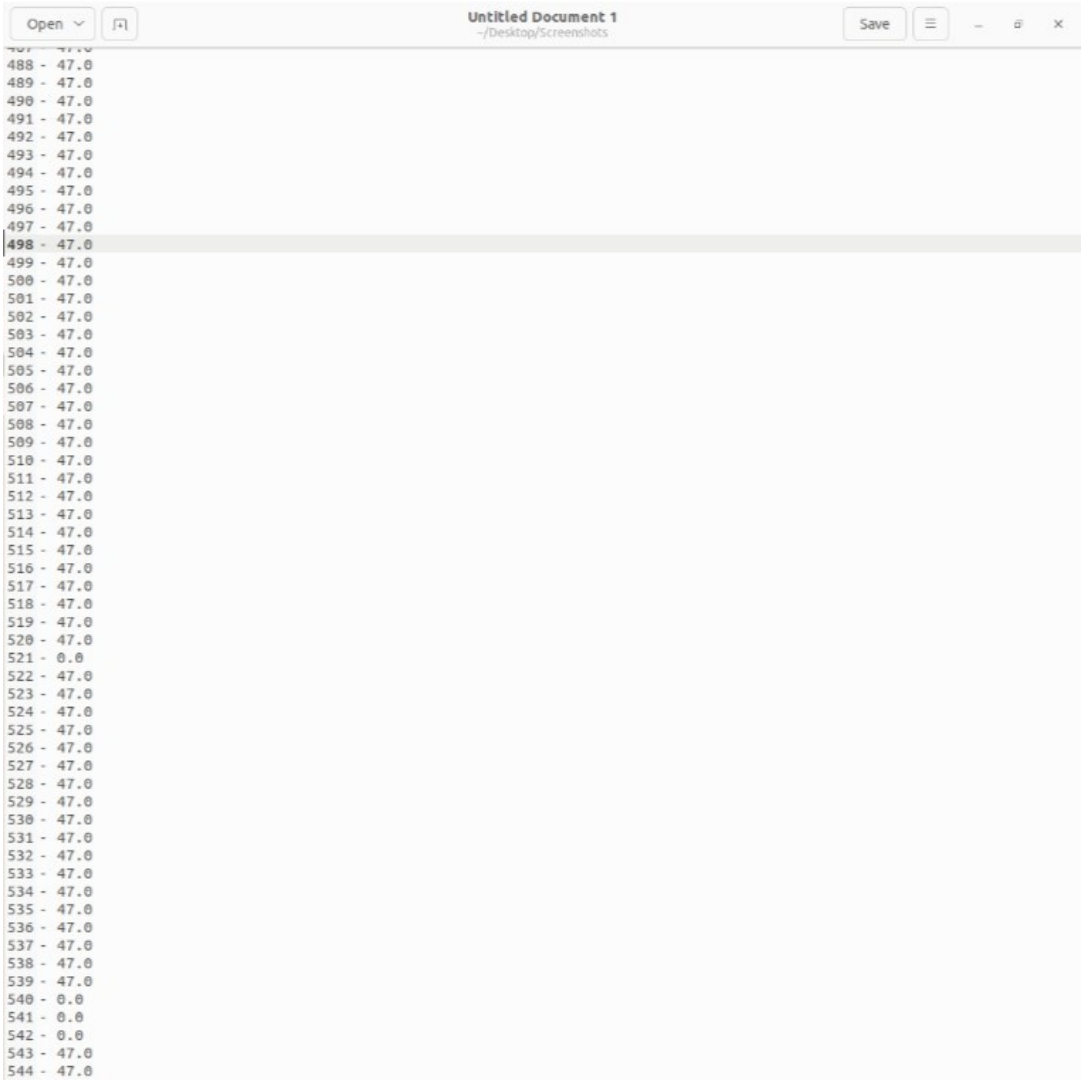


Abbildung 3: Visualisierung der Sensordaten im Terminal

Nach Abschluss der Kartierung wurde die autonome Navigation des TurtleBot4 getestet. Hierfür konnten in RViz Zielpositionen innerhalb der erstellten Karte festgelegt werden, welche der Roboter anschließend selbstständig anfuhr. Die Navigationssoftware berechnete dabei anhand der Karte und der aktuellen Sensordaten eine geeignete Fahrstrecke und passte die Bewegungsbefehle während der Fahrt an.

In der RViz Ansicht wurden zusätzlich verschiedene farbige Bereiche dargestellt. Diese Karte aus Abbildung 4 berücksichtigt die erkannten Hindernisse und deren Umgebung bei der Wegplanung. Die dunklen Bereiche kennzeichnen Hindernisse beziehungsweise nicht befahrbare Flächen, während die farbigen Zonen um diese Hindernisse erhöhte Bewegungskosten darstellen. Je näher sich eine Position an einem Hindernis befindet, desto ungünstiger wird sie für die Routenplanung bewertet. Dadurch soll ein ausreichender Sicherheitsabstand eingehalten werden.

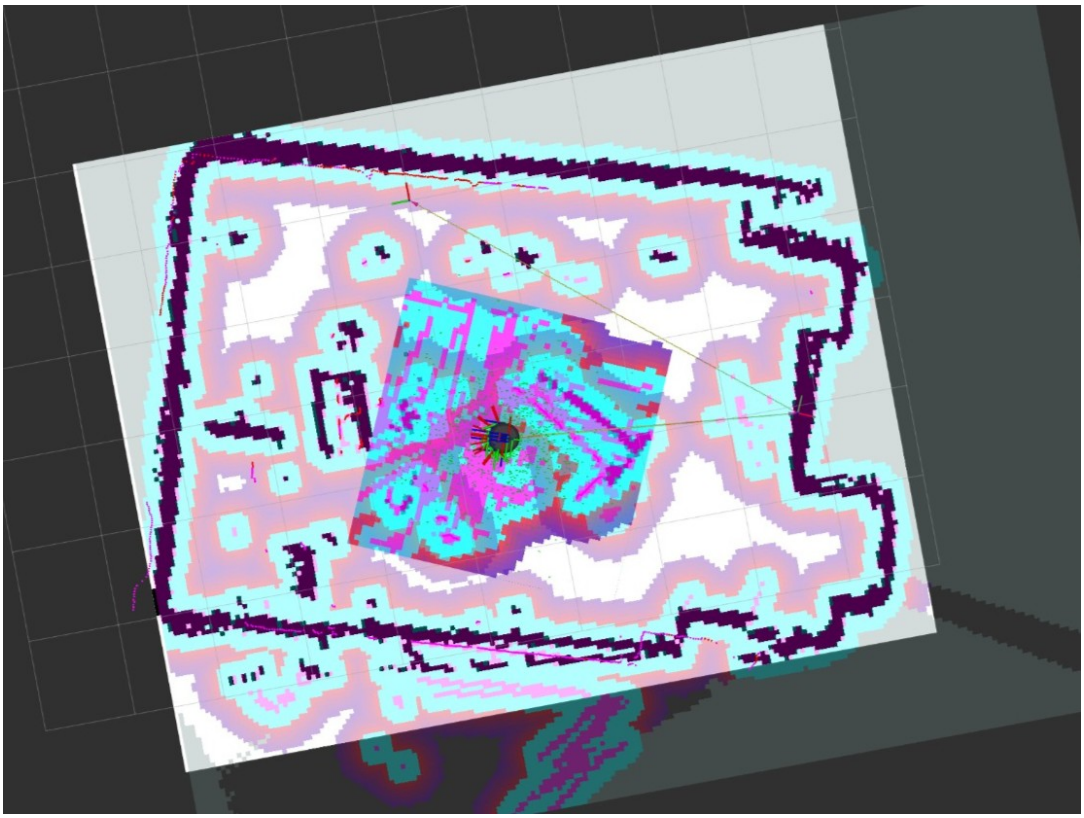


Abbildung 4: Autonome Navigierung im Raum

Während der autonomen Fahrt fiel auf, dass sich um die aktuelle Position des Roboters ein auffälliger quadratischer Bereich bildete, der sich gemeinsam mit diesem durch die Karte bewegte. Die ungewöhnliche Darstellung hatte jedoch keinen erkennbaren Einfluss auf das Fahrverhalten, sodass die Navigation weiterhin funktionierte.

Eine weitere Beobachtung betraf die Wahl der Zielposition. Wurde diese zu nah an einem Hindernis platziert, hatte der Roboter Schwierigkeiten, den gewünschten Punkt zu erreichen. In einem Fall stoppte er vollständig und der LED Ring leuchtete rot auf. Dies deutete auf einen Fehler- oder Sicherheitszustand hin, dessen genaue Ursache jedoch nicht festgestellt werden konnte.

Im letzten Teil der Praktikumsaufgabe sollte eine eigene Anwendung für den TurtleBot4 entwickelt werden. Zunächst wurde hierfür eine angepasste Variante der Hindernisvermeidung aus Aufgabe 2.4 in Betracht gezogen. Diese sollte die Infrarotsensoren des TurtleBot4 zur Hinderniserkennung und die Odometrie zur Überwachung der Drehbewegung verwenden. Da dieser Ansatz jedoch weitgehend dem bereits umgesetzten Bewegungsablauf entsprach, wurde stattdessen ein neues Programm zur autonomen Wandverfolgung entwickelt. Dadurch konnten die erweiterten Möglichkeiten des integrierten LiDAR Scanners besser genutzt werden.

Hierfür wurde eine Python Node mit dem Namen ‘wall_follower‘ erstellt. Diese verwendet die ROS 2 Bibliothek ‘rclpy‘ und abonniert das Topic ‘/scan‘, über welches die Messwerte des LiDAR-Scanners als Nachrichten des Typs ‘sensor_msgs/msg/LaserScan‘ bereitgestellt werden. Die Bewegungsbefehle werden über einen Publisher auf dem Topic ‘/cmd_vel‘ als Nachrichten des Typs ‘geometry_msgs/msg/Twist‘ veröffentlicht.

Ziel des Programms ist es, den TurtleBot4 entlang einer Wand auf seiner rechten Seite zu bewegen und dabei einen möglichst konstanten Abstand von 0,40 m einzuhalten. Zusätzlich wurde ein Mindestabstand von 0,30 m sowie ein Grenzwert von 0,45 m zur Erkennung von Hindernissen in Fahrtrichtung festgelegt.

Zur Abstandsermittlung werden die LiDAR Messwerte in verschiedenen Richtungen ausgewertet. Hierfür wurde eine Funktion implementiert, die einen Winkelbereich von jeweils $\pm 5°$ um die ge-

wünschte Messrichtung betrachtet. Ungültige Messwerte werden ausgeschlossen und anschließend der kleinste gültige Abstand bestimmt. Die Zuordnung der Winkel zu den jeweiligen Fahrtrichtungen wurde dabei an die tatsächliche Einbaulage des LiDAR Scanners angepasst.

```
61      def get_distance(self, scan, angle_deg):
62
63          angle_rad = math.radians(angle_deg)
64
65          #  5  Grad um den gewuenschten Winkel
66          angle_min = angle_rad - math.radians(5)
67          angle_max = angle_rad + math.radians(5)
68
69          index_min = int(
70              (angle_min - scan.angle_min) / scan.angle_increment
71          )
72
73          index_max = int(
74              (angle_max - scan.angle_min) / scan.angle_increment
75          )
76
77          index_min = max(0, index_min)
78          index_max = min(len(scan.ranges) - 1, index_max)
79
80          distances = []
81
82          for i in range(index_min, index_max + 1):
83
84              distance = scan.ranges[i]
85
86              if math.isfinite(distance):
87                  if scan.range_min <= distance <= scan.range_max:
88                      distances.append(distance)
89
90          if not distances:
91              return float('inf')
92
93          return min(distances)
```

Quellcode 12: Auswertung der LiDAR Messwerte zur Abstandsermittlung (Anhang E)

Während der Entwicklung und Erprobung des Programms traten zunächst Schwierigkeiten bei der Zuordnung der LiDAR Messwerte zu den jeweiligen Fahrtrichtungen auf. Nach einer längeren Fehlersuche stellte sich heraus, dass der LiDAR Scanner um 180° verdreht montiert war. Dadurch entsprach die tatsächliche Orientierung der Messwerte nicht der ursprünglich angenommenen Ausrichtung. Die Winkel für vorne, hinten, rechts und links waren somit um 180° verschoben, wodurch die Abstandsmessungen zunächst den falschen Richtungen zugeordnet wurden.

Nachdem die Ursache erkannt worden war, wurden die entsprechenden Winkel im Python Code an die tatsächliche Ausrichtung des Sensors angepasst. Anschließend konnten die Entfernungen korrekt den jeweiligen Richtungen zugeordnet werden, sodass die Wandverfolgung wie vorgesehen funktionierte. Die Fehlersuche nahm einen erheblichen Teil der Entwicklungszeit in Anspruch und verdeutlichte, wie wichtig die korrekte Berücksichtigung der Sensororientierung bei der Verarbeitung von Messdaten ist.

Die Bewegungssteuerung erfolgt über einen periodischen Timer mit einem Intervall von 0,1 Sekunden. Bei jedem Aufruf werden die zuletzt empfangenen LiDAR Daten ausgewertet und entsprechende Geschwindigkeitsbefehle erzeugt. Liegen noch keine Messwerte vor, wird zunächst ein Stoppbefehl veröffentlicht.

Erkennt der Roboter ein Hindernis in Fahrtrichtung mit einem Abstand von weniger als 0,45 m, wird die Vorwärtsbewegung gestoppt und eine Linksdrehung mit einer Winkelgeschwindigkeit von 0,8 rad/s eingeleitet. Ist der Weg frei, wird die Fahrtrichtung anhand des gemessenen Abstands zur rechten Wand angepasst.

```
        if front_distance < self.front_obstacle_distance:

            self.get_logger().info(
                f'Hindernis vorne: {front_distance:.2f} m - '
                'drehe links'
            )

            cmd.linear.x = 0.0
            cmd.angular.z = self.turn_speed

            self.cmd_publisher.publish(cmd)
            return
```

Quellcode 13: Hinderniserkennung und Einleitung einer Linksdrehung (Anhang E)

Hierfür wurde eine proportionale Regelung vorgesehen. Die Abweichung zwischen dem Sollabstand von 0,40 m und dem gemessenen Wandabstand wird mit einem Proportionalfaktor von 1,5 multipliziert, um daraus eine Winkelgeschwindigkeit abzuleiten. Diese wird auf einen Betrag von maximal 0,8 rad/s begrenzt. Die vorgesehene Vorwärtsgeschwindigkeit beträgt 0,15 m/s. Unterschreitet der Wandabstand 0,30 m, wird die Geschwindigkeit auf 60 % reduziert und eine stärkere Korrektur nach links vorgenommen.

```
        if right_distance < self.min_distance:

            error = self.target_distance - right_distance

            angular = self.kp * error

            angular = max(
                -self.max_angular_speed,
                min(self.max_angular_speed, angular)
            )

            cmd.linear.x = self.forward_speed * 0.6
            cmd.angular.z = angular

        else:

            # Fehler:
            #
            # positive Zahl = Wand zu weit weg
            # negative Zahl = Wand zu nah

            error = self.target_distance - right_distance

            # Bei einer rechten Wand:
            #
            # Wand zu weit weg  -> nach rechts drehen
            # Wand zu nah        -> nach links drehen

            angular = -self.kp * error

            angular = max(
                -self.max_angular_speed,
                min(self.max_angular_speed, angular)
            )

            cmd.linear.x = self.forward_speed
            cmd.angular.z = angular

        self.cmd_publisher.publish(cmd)
```

Quellcode 14: Berechnung der Bewegungsbefehle anhand des Wandabstands (Anhang E)

Im Unterschied zur Hindernisvermeidung aus Aufgabe 2.4 basiert dieser Ansatz auf einer fortlaufenden Abstandsmessung und der daraus berechneten Anpassung der Fahrtrichtung. Nach der Korrektur der Sensororientierung konnte das Programm erfolgreich getestet werden und der TurtleBot4 einer Wand folgen. Damit wurde eine Bewegungssteuerung umgesetzt, die ohne vorherige Kartierung auskommt und ihre Fahrtrichtung anhand der aktuellen Sensormesswerte anpasst.

2.6 Auswertung

Im Rahmen des Praktikums konnten die grundlegenden Funktionen von ROS 2 erfolgreich auf zwei unterschiedlichen Robotersystemen angewendet werden. Sowohl die Hindernisvermeidung des selbst aufgebauten ESP32-Roboters als auch die Wandverfolgung mit dem TurtleBot4 wurden implementiert und praktisch getestet.

Bei der Hindernisvermeidung erkannte der ESP32 Roboter mithilfe des Ultraschallsensors Hindernisse und führte anschließend selbstständig eine Ausweichbewegung aus. Dabei wurde die Drehbewegung durch die Messwerte des Gyroskops überwacht. Trotz kleiner Abweichungen während der Rückwärtsfahrt konnte die vorgesehene Bewegungsabfolge erfolgreich umgesetzt werden.

Auch der entwickelte Wall Follower für den TurtleBot4 funktionierte nach der Anpassung der LiDAR Winkelzuordnung wie vorgesehen. Der Roboter konnte der Wand auf seiner rechten Seite selbstständig folgen und seine Fahrtrichtung anhand der kontinuierlich erfassten Abstandswerte anpassen. Die zunächst fehlerhafte Orientierung des LiDAR Scanners führte zu einer längeren Fehlersuche, verdeutlichte jedoch gleichzeitig die Bedeutung einer korrekten Sensorkonfiguration.

Die erfolgreiche Funktionsweise beider Anwendungen wurde zusätzlich durch Videoaufnahmen dokumentiert. Diese können über die nachfolgend eingefügten Links aufgerufen werden:

ESP32-Roboter – Hindernisvermeidung:
https://youtube.com/shorts/JqkiD1h4NuY?feature=share

TurtleBot4 – Wall-Follower:
https://youtu.be/RsvQyWxunuw

Insgesamt bot das Praktikum einen praxisnahen Einstieg in die mobile Robotik und die Verwendung von ROS 2. Besonders das Publisher-Subscriber-Prinzip und die einheitliche Kommunikation über Topics erwiesen sich als hilfreich, da sich damit unterschiedliche Hardwareplattformen nach einem ähnlichen Prinzip ansteuern lassen. Während beim ESP32 Roboter zunächst die Motorsteuerung und die Kommunikation über micro-ROS eingerichtet werden mussten, konnten beim TurtleBot4 bereits vorhandene Funktionen und Sensordaten direkt genutzt werden.

3 Teil 2: Programmieraufgaben Echtzeitsysteme

Die für diesen Teil entwickelten Quellcodes und kompilierten Programme sind im GitHub-Repository im Verzeichnis `Programmieraufgabe` verfügbar. Die Quellcodes sind an den relevanten Stellen kommentiert. Um die Dokumentation übersichtlich zu halten, wird auf die Darstellung von Codeabschnitten für diesen Teil verzichtet.

3.1 Sockets

In dieser Aufgabe geht es um die Erstellung und Verwendung von Sockets zur Kommunikation zwischen Programmen über ein Netzwerk. Diese Form der Kommunikation bildet die Grundlage zahlreicher moderner IT-Dienste.
Im Rahmen dieser Aufgabe sind mehrere Programme zu implementieren.

3.1.1 Teilaufgabe a: Server

Der Server verwendet TCP/IPv4 und bindet an `127.0.0.1:5000`. Eingehende Verbindungen werden mit `accept()` angenommen und nacheinander bearbeitet. Pro Verbindung wird eine Anfrage verarbeitet. anschließend wird die Verbindung geschlossen.

Protokoll Anfragen und Antworten enden mit LF (*Line Feed* bzw. Zeilenumbruch). Die maximale Anfragelänge beträgt 128 Byte einschließlich LF. Kommandos werden unter Beachtung der Groß- und Kleinschreibung ausgewertet.

Anfrage	**Antwort**
`PING`	`OK PONG`
Unbekanntes Kommando	`ERR UNKNOWN_COMMAND`
Längengrenze überschritten	`ERR LINE_TOO_LONG`

Unvollständige Anfragen bei Verbindungsende werden verworfen. Der Empfang berücksichtigt die fehlenden Nachrichtengrenzen von TCP. eine Sendeschleife behandelt Teilübertragungen. Systemaufrufe werden auf Fehler geprüft. Die sequenzielle Verarbeitung kann durch einen wartenden Client blockiert werden.

Test 1 Der TCP-Verbindungsaufbau wurde mit `nc -vz 127.0.0.1 5000` erfolgreich geprüft.

Test 2 Der Verbindungsaufbau wurde mit Netcat erfolgreich geprüft. Die Anfrage `PING` wurde mit `OK PONG` beantwortet. Ein unbekanntes Kommando wurde mit `ERR UNKNOWN_COMMAND` abgewiesen. 128 Zeichen ohne abschließendes LF führten erwartungsgemäß zu `ERR LINE_TOO_LONG`. Der Server nahm anschließend weitere Verbindungen an.

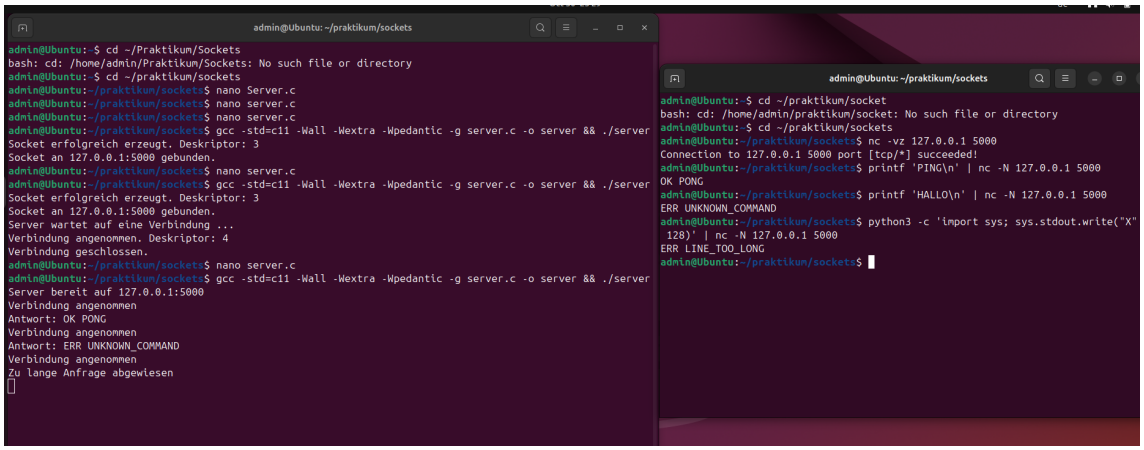


Abbildung 5: Server antwortet auf Anfragen aus einem zweiten Terminal

3.1.2 Teilaufgabe b: Client

Der Client übernimmt IPv4-Adresse, Port und Kommando als Programmargumente. Nach Prüfung der Eingaben wird mit `connect()` eine TCP-Verbindung zum Server aufgebaut.

Das Kommando wird um LF ergänzt und vollständig gesendet. Die Antwort wird bis zum abschließenden LF empfangen und ausgegeben. Anschließend wird der Socket geschlossen. Das Protokoll entspricht Teilaufgabe a.

Ungültige Eingaben, Verbindungsfehler und unvollständige Antworten werden erkannt und gemeldet. Sende- und Empfangsschleifen berücksichtigen Teilübertragungen.

Tests Die Anfrage `PING` wurde mit `OK PONG` beantwortet. Das unbekannte Kommando `HALLO` führte zu `ERR UNKNOWN_COMMAND`. Die Portangabe `abc` wurde vom Client vor dem Verbindungsaufbau als ungültig zurückgewiesen. Bei beendetem Server wurde der Verbindungsversuch mit `Connection refused` abgewiesen und vom Client als Fehler gemeldet.

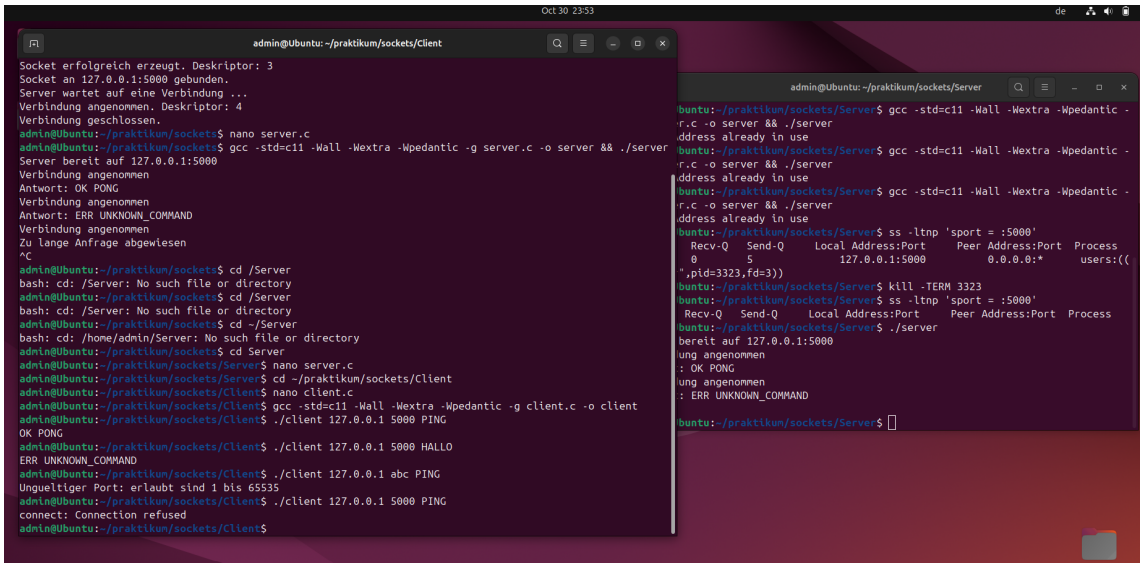


Abbildung 6: Server-Client-Kommunikation zwischen beiden Terminals

3.1.3 Teilaufgabe c: Verwaltung einer gemeinsamen Liste

Der Server verwaltet einen gemeinsamen Nachrichtenverlauf als Array mit maximal 20 Einträgen. Jede Nachricht enthält höchstens 100 druckbare ASCII-Zeichen. Die Einträge werden fortlaufend ab 1 nummeriert und bleiben bis zum Ende des Serverprozesses erhalten.

Kommando	Funktion / Antwort
`ADD <Text>`	Anhängen: `OK ADDED <Nr.>`
`COUNT`	Anzahl: `OK COUNT <Anzahl>`
`GET <Nr.>`	Lesen: `OK MSG <Nr.> <Text>`

Die LF-Trennung und die PDU-Grenze von 128 Byte bleiben bestehen. Ungültige Nachrichtentexte, ungültige Indizes und eine volle Liste werden mit `ERR BAD_MESSAGE`, `ERR BAD_INDEX` beziehungsweise `ERR LIST_FULL` beantwortet. Nicht erlaubte Zeichen führen zu `ERR BAD_REQUEST`.

Alle Clients greifen auf denselben serverseitigen Zustand zu. Da Anfragen sequenziell verarbeitet werden, sind keine zusätzlichen Sperren erforderlich.

Tests Die zunächst leere Liste wurde über getrennte Clientaufrufe um zwei Nachrichten erweitert. Anschließend lieferte `COUNT` den Wert 2. beide Einträge konnten mit `GET` korrekt gelesen werden. Ein nicht vorhandener Index wurde mit `ERR BAD_INDEX`, ein leerer Nachrichtentext mit `ERR BAD_MESSAGE` abgewiesen.

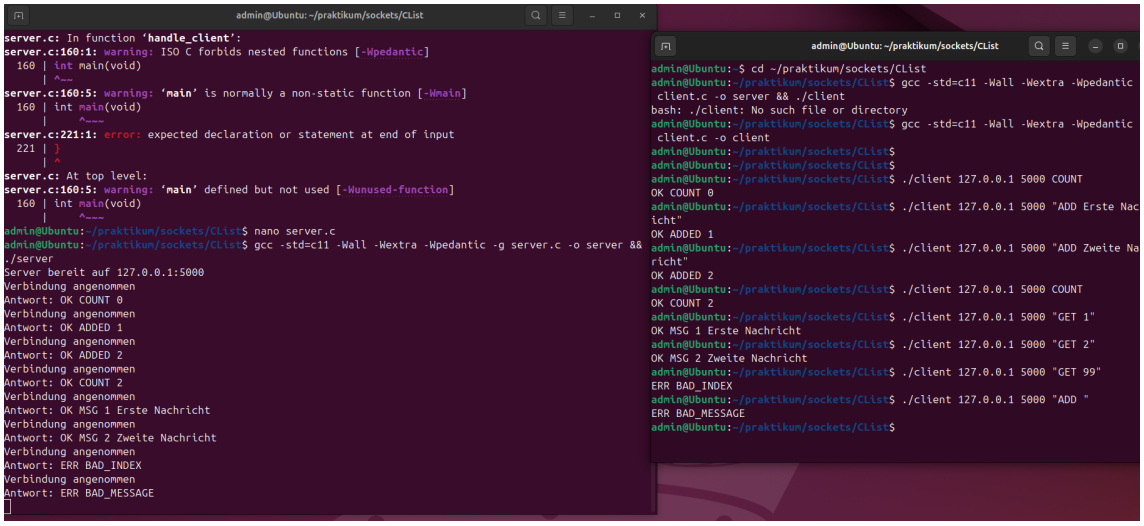


Abbildung 7: Server-Client-Kommunikation zwischen beiden Terminals mit einer gemeinsamen Liste

3.1.4 Teilaufgabe d: Peer-to-Peer-Erweiterung

Beide Instanzen verwenden dasselbe Programm und lauschen auf unterschiedlichen TCP-Ports. Jede Instanz kann Anfragen senden und empfangen. Dazu verarbeitet der Hauptthread Konsoleneingaben und ausgehende Anfragen, während ein Empfangsthread eingehende Verbindungen bedient.

Die Kommandos aus Teilaufgabe c werden weiterverwendet. Jeder Peer besitzt eine eigene Nachrichtenliste. Eine mit `ADD` gesendete Nachricht wird nur beim angesprochenen Peer gespeichert und nicht automatisch auf andere Peers kopiert.

Innerhalb eines Peers bearbeitet ausschließlich der Empfangsthread die Liste. Da er die Anfragen nacheinander verarbeitet, ist keine zusätzliche Sperre für die Liste erforderlich.

Beim Beenden setzt der Hauptthread ein Stoppsignal, das der Empfangsthread sicher abfragen kann. Der Hauptthread wartet anschließend, bis der Empfangsthread beendet ist.

Der Empfang einer vollständigen PDU ist auf fünf Sekunden begrenzt. Verbindungs- und Sendewartezeiten werden ebenfalls begrenzt. Fehler eines Partners werden gemeldet. die eigene Instanz bleibt nutzbar. Schreibanfragen werden nicht automatisch wiederholt: Bei verlorener Antwort kann unklar sein, ob der Eintrag bereits gespeichert wurde. Die Listen gehen beim Beenden verloren.

Tests Zwei Instanzen auf den Ports 5001 und 5002 beantworteten gegenseitige `PING`-Anfragen. Beide konnten Nachrichten beim jeweils anderen Peer speichern. Das anschließende Lesen der eigenen Listen lieferte jeweils die Nachricht des Kommunikationspartners.

Nach dem Beenden von Peer B meldete Peer A bei einem weiteren Verbindungsversuch `Connection refused`. Die eigene Liste blieb weiterhin abfragbar (`OK COUNT 1`).

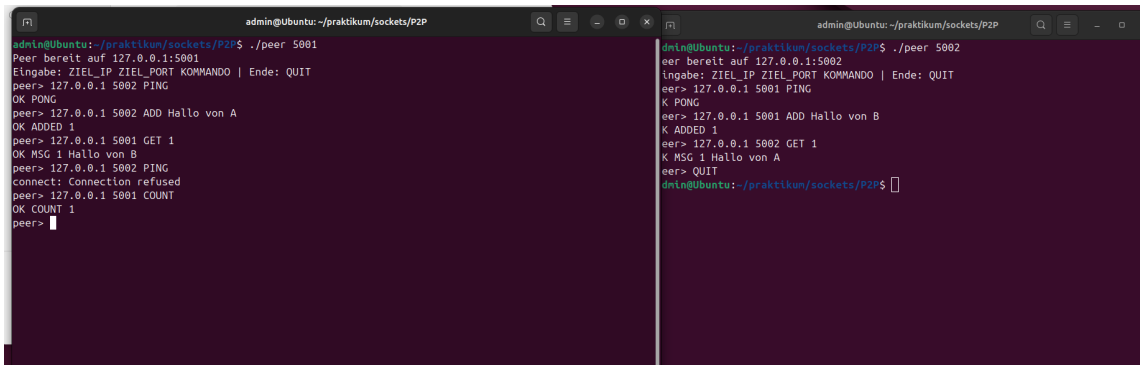


Abbildung 8: Peer-2-Peer-Kommunikation zwischen zwei Terminals

3.2 Interprozesskommunikation (IPC)

Ausgehend von den Grundlagen aus der Vorlesung machen Sie sich mit den Funktionsweisen verschiedener IPC-Varianten vertraut und entscheiden Sie sich für eine geeignete Methode zur Durchführung der folgenden Aufgaben. Mögliche IPC-Mechanismen sind beispielsweise:

- Shared Memory
- Message Queues
- Semaphoren
- Pipes oder Named Pipes (FIFOs)

Für die IPC-Aufgaben werden System-V-Message-Queues gewählt, da sie strukturierte Anfragen und Antworten zwischen Prozessen übertragen und dabei Nachrichtengrenzen erhalten. Die gemeinsame Nachrichtenliste verbleibt im zuständigen Prozess, sodass bei sequenzieller Verarbeitung keine zusätzlichen Sperren für die Liste erforderlich sind. Für die spätere gleichberechtigte Kommunikation kann jede Instanz eine eigene Empfangsqueue verwalten und zugleich Nachrichten an andere Instanzen senden.

3.2.1 Teilaufgabe a: Server-Prozess

Der Server erstellt eine System-V-Message-Queue mit dem Schlüssel `0x45524d01` und den Zugriffsrechten `0600`. Eine bereits vorhandene Queue verhindert den Start einer zweiten Serverinstanz.

Die PDU enthält den Nachrichtentyp, die ID der Rückantwortqueue, eine Anfragenummer und ein Textfeld mit 128 Byte. Typ 1 bezeichnet Anfragen, Typ 2 Antworten. Die Anfragenummer wird

unverändert zurückgegeben. Der Text ist nullterminiert. ein LF-Trennzeichen entfällt, da Message Queues Nachrichtengrenzen erhalten.

`PING` wird mit `OK PONG` beantwortet. Unbekannte Kommandos und ungültige Texte erzeugen `ERR UNKNOWN_COMMAND` beziehungsweise `ERR BAD_REQUEST`. Falsche Nachrichtentypen oder PDU-Größen werden verworfen.

Die Verarbeitung erfolgt sequenziell mit einer Pause von 100 ms bei leerer Queue. Antworten werden nichtblockierend versendet. Bei SIGINT oder SIGTERM wird die eigene Anfragequeue entfernt.

Tests Die Anfragequeue wurde mit `ipcs -q` nachgewiesen. Eine zweite Serverinstanz wurde abgewiesen. nach SIGINT wurde die Queue entfernt. Der Nachrichtenaustausch wird zusammen mit dem Client aus Teilaufgabe b geprüft.

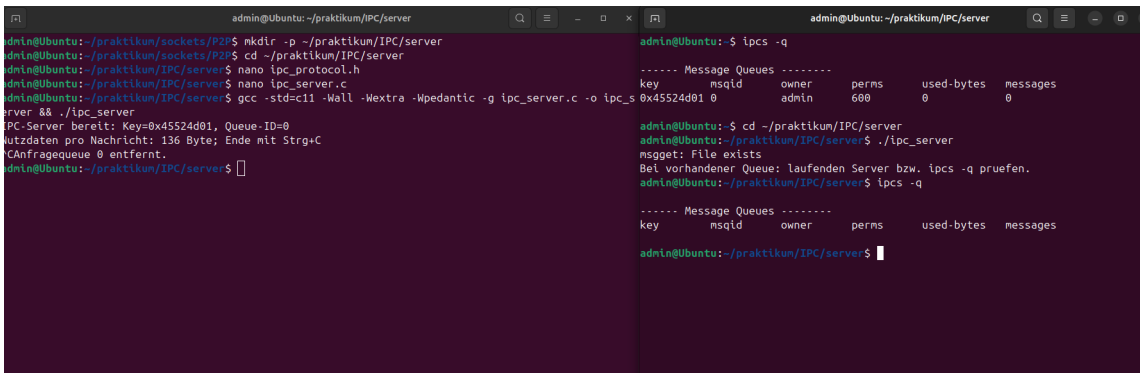


Abbildung 9: Nachweis der Anfragequeue des Server-Prozesses

3.2.2 Teilaufgabe b: Client-Prozess

Der Client übernimmt ein Kommando als Programmargument und öffnet die vorhandene Serverqueue mit `msgget()`. Für die Antwort erstellt er eine eigene Message Queue. Deren ID wird zusammen mit dem Kommando und einer Anfragenummer als PDU vom Typ 1 an den Server gesendet.

Anschließend empfängt der Client eine Antwort vom Typ 2, prüft deren Größe und Zuordnung und gibt den Text aus. Die Wartezeit ist begrenzt. Abschließend wird die eigene Rückantwortqueue entfernt. die Serverqueue bleibt bestehen. Server und Client verwenden denselben Protokoll-Header.

Funktionstest zu a und b Der Client sendete nacheinander die Kommandos `PING` und `HALLO`. Der Server beantwortete diese mit `OK PONG` beziehungsweise `ERR UNKNOWN_COMMAND`. Die Antworten wurden über separate Rückantwortqueues an die jeweiligen Clientprozesse übermittelt und korrekt ausgegeben.

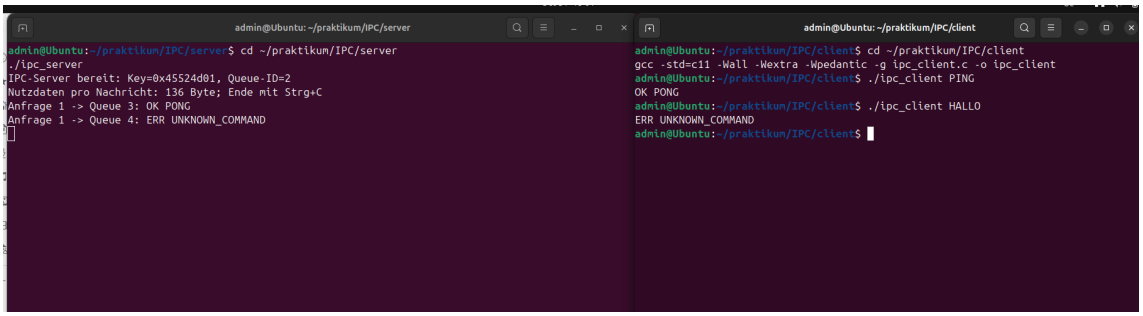


Abbildung 10: Datenaustausch zwischen beiden Terminals

3.2.3 Teilaufgabe c: Verwaltung einer gemeinsamen Liste

Der IPC-Server verwaltet einen gemeinsamen Nachrichtenverlauf als Array mit maximal 20 Einträgen zu jeweils höchstens 100 druckbaren ASCII-Zeichen. Das bestehende Protokoll wird um drei Kommandos erweitert:

Kommando	Funktion / Antwort
`ADD <Text>`	Anhängen: `OK ADDED <Nr.>`
`COUNT`	Anzahl: `OK COUNT <Anzahl>`
`GET <Nr.>`	Lesen: `OK MSG <Nr.> <Text>`

Ungültige Indizes, leere oder überlange Nachrichtentexte und eine volle Liste werden mit Fehlerantworten abgewiesen. Der Client aus Teilaufgabe b kann unverändert verwendet werden.

Die Liste befindet sich ausschließlich im Serverprozess. Clients greifen über Message Queues darauf zu. Da der Server Anfragen sequenziell verarbeitet, erfolgen keine gleichzeitigen Zugriffe auf die Datenstruktur. zusätzliche Semaphoren sind daher nicht erforderlich. Die Einträge bleiben bis zum Ende des Serverprozesses erhalten.

Funktionstest Die zunächst leere Liste wurde durch zwei getrennte Clientaufrufe erweitert. Anschließend lieferte `COUNT` den Wert 2. Beide Nachrichten konnten über ihre Nummern korrekt gelesen werden. Der Zugriff auf den nicht vorhandenen Eintrag 99 wurde mit `ERR BAD_INDEX` abgewiesen.

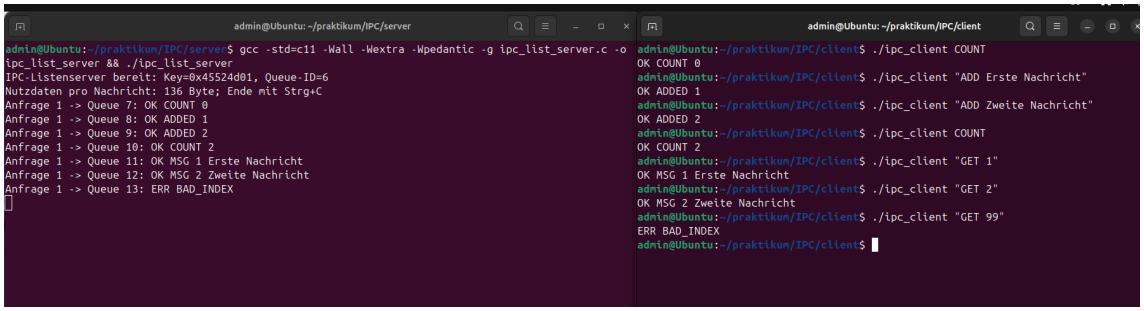


Abbildung 11: Datenaustausch zwischen beiden Terminals mit einer gemeinsamen Liste

3.2.4 Teilaufgabe d: Gleichberechtigte Kommunikation

Die feste Trennung zwischen Server und Client wird durch ein gemeinsames Peer-Programm aufgehoben. Jede Instanz besitzt eine eindeutige Peer-ID und eine eigene Anfragequeue. Deren Schlüssel ergibt sich aus `0x45525000` zuzüglich der Peer-ID. Das PDU-Format bleibt unverändert.

Der Hauptthread verarbeitet Konsoleneingaben und sendet Anfragen. Ein Empfangsthread bearbeitet unabhängig davon eingehende Anfragen. Antworten werden über temporäre Rückantwortqueues zugeordnet. Nichtblockierendes Senden und eine begrenzte Antwortwartezeit verhindern dauerhaftes Warten auf volle Queues oder ausgefallene Partner.

Jeder Peer verwaltet eine eigene, nicht replizierte Liste. Nur der Empfangsthread greift darauf zu. Listenoperationen werden daher sequenziell ausgeführt. Ein atomares Stoppsignal koordiniert das Beenden der Threads. Beim Beenden werden die eigenen Queues entfernt. Schreibanfragen werden nicht automatisch wiederholt, da eine verlorene Antwort trotz erfolgter Speicherung auftreten kann.

Funktionstest Zwei Peers beantworteten gegenseitige `PING`-Anfragen und speicherten Nachrichten beim jeweils anderen Peer. Beim Lesen der eigenen Listen wurde jeweils die Nachricht des Kommunikationspartners ausgegeben. Fehlerhafte Eingaben wurden abgewiesen.

Nach dem Beenden von Peer 2 wurde dessen Anfragequeue entfernt. Peer 1 meldete den nicht erreichbaren Partner und konnte anschließend weiterhin seine eigene Liste abfragen (`OK COUNT 1`).

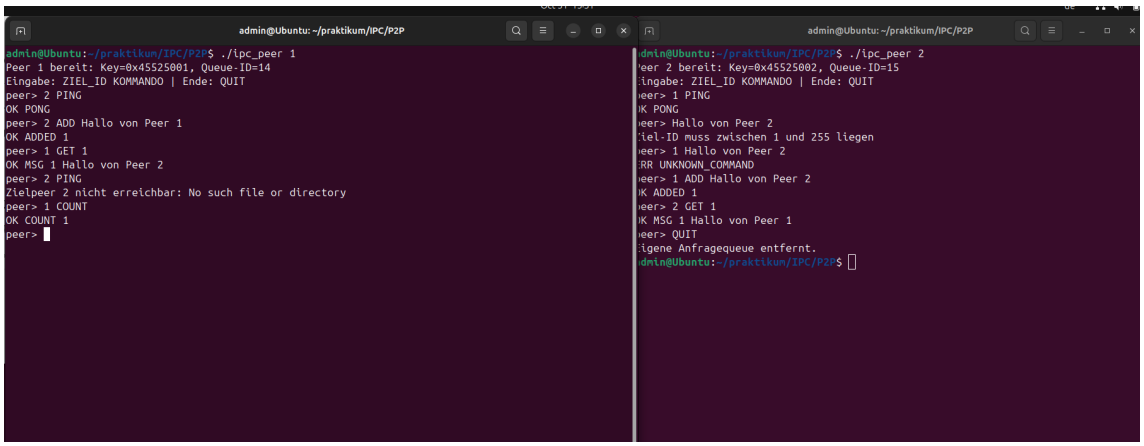


Abbildung 12: Gleichberechtigte Kommunikation zwischen beiden Terminals

3.3 Auswertung

Die Aufgaben zeigen den Aufbau einer Anfrage-Antwort- Kommunikation über TCP-Sockets und System-V-Message-Queues. Bei TCP mussten Nachrichtengrenzen ausdrücklich definiert werden, während Message Queues diese bereits erhalten. Konsistente Listenzugriffe wurden durch die sequenzielle Verarbeitung im jeweils zuständigen Prozess erreicht. Die Peer-Erweiterungen ermöglichen durch getrennte Sende- und Empfangsabläufe eine gleichberechtigte Kommunikation. Die Listen werden ausschließlich im Arbeitsspeicher des jeweiligen Prozesses gespeichert und gehen beim Beenden verloren.

Die durchgeführten Tests bestätigten den Nachrichtenaustausch sowie das Speichern und Lesen von Listeneinträgen. Fehlerhafte Anfragen und nicht erreichbare Kommunikationspartner wurden erkannt und durch entsprechende Meldungen angezeigt. Dabei wurde deutlich, dass neben der eigentlichen Datenübertragung auch die Fehlerbehandlung und das Freigeben von Ressourcen wesentliche Bestandteile der Implementierung sind.

Appendix

A shape_controller.py

/home/vboxuser/ros2_ws/src/turtle_controller/turtle_controller

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist


class ShapeController(Node):

    def __init__(self):
        super().__init__('shape_controller')

        self.publisher_ = self.create_publisher(
            Twist,
            '/cmd_vel',
            10
        )

        self.timer_period = 0.1
        self.timer = self.create_timer(
            self.timer_period,
            self.timer_callback
        )


        self.state = 0

        self.counter = 0

        self.side = 0

        self.get_logger().info('Shape Controller gestartet')


    def timer_callback(self):

        msg = Twist()


        if self.state == 0:

            msg.linear.x = 1.0
            msg.angular.z = 0.0

            self.counter += 1

            if self.counter >= 20:

                self.counter = 0
                self.state = 1

        elif self.state == 1:

            msg.linear.x = 0.0
            msg.angular.z = 1.57

            self.counter += 1

            if self.counter >= 10:

                self.counter = 0
                self.state = 0

                self.side += 1

                self.get_logger().info(
```

```
65                      f'Seite {self.side} fertig'
66                  )
67
68          if self.side >= 4:
69
70              msg.linear.x = 0.0
71              msg.angular.z = 0.0
72
73          self.publisher_.publish(msg)
74
75
76  def main(args=None):
77
78      rclpy.init(args=args)
79
80      node = ShapeController()
81
82      rclpy.spin(node)
83
84      node.destroy_node()
85
86      rclpy.shutdown()
87
88
89  if __name__ == '__main__':
90      main()
```

Quellcode 15: Implementierung der Shape Controller Node

B 2.3 C++ Code

```
#include <Arduino.h>
#include <micro_ros_platformio.h>

#include <rcl/rcl.h>
#include <rclc/rclc.h>
#include <rclc/executor.h>

#include <geometry_msgs/msg/twist.h>

#if !defined(MICRO_ROS_TRANSPORT_ARDUINO_WIFI)
#error This example is only available for Arduino framework with wifi transport.
#endif

char ssid[] = "WLAN-Name";
char psk[]  = "Passwort";

IPAddress agent_ip(192, 168, 2, 218);
size_t agent_port = 8888;

#define MOTOR_TX_PIN 17
#define START_BYTE   0xAA

#define LEFT_MOTOR_ID  1
#define RIGHT_MOTOR_ID 2

#define MODE_CONTINUOUS 1
#define MOTOR_PARAM     3

HardwareSerial MotorSerial(2);


void sendMotorCommand(
    uint8_t id,
    uint8_t mode,
    uint8_t dir,
    uint16_t param)
{
    MotorSerial.write(START_BYTE);
    MotorSerial.write(id);
    MotorSerial.write(mode);
    MotorSerial.write(dir);

    MotorSerial.write((uint8_t)((param >> 8) & 0xFF));
    MotorSerial.write((uint8_t)(param & 0xFF));

    MotorSerial.flush();
}

void driveForward()
{
// Links vorwaerts
    sendMotorCommand(
        LEFT_MOTOR_ID,
        MODE_CONTINUOUS,
        1,
        MOTOR_PARAM
    );

// Rechts vorwaerts
    sendMotorCommand(
        RIGHT_MOTOR_ID,
        MODE_CONTINUOUS,
        0,
        MOTOR_PARAM
    );
}

void driveBackward()
{
    // Links rueckwaerts
```

```
71      sendMotorCommand(
72          LEFT_MOTOR_ID,
73          MODE_CONTINUOUS,
74          0,
75          MOTOR_PARAM
76      );
77
78      // Rechts rueckwaerts
79      sendMotorCommand(
80          RIGHT_MOTOR_ID,
81          MODE_CONTINUOUS,
82          1,
83          MOTOR_PARAM
84      );
85  }
86
87  void turnLeft()
88  {
89      // Linkes Rad rueckwaerts
90      sendMotorCommand(
91          LEFT_MOTOR_ID,
92          MODE_CONTINUOUS,
93          0,
94          MOTOR_PARAM
95      );
96
97      // Rechtes Rad vorwaerts
98      sendMotorCommand(
99          RIGHT_MOTOR_ID,
100         MODE_CONTINUOUS,
101         0,
102         MOTOR_PARAM
103     );
104 }
105
106 void turnRight()
107 {
108     // Linkes Rad vorwaerts
109     sendMotorCommand(
110         LEFT_MOTOR_ID,
111         MODE_CONTINUOUS,
112         1,
113         MOTOR_PARAM
114     );
115
116     // Rechtes Rad rueckwaerts
117     sendMotorCommand(
118         RIGHT_MOTOR_ID,
119         MODE_CONTINUOUS,
120         1,
121         MOTOR_PARAM
122     );
123 }
124
125 rcl_subscription_t subscriber;
126
127 geometry_msgs__msg__Twist msg;
128
129 rclc_executor_t executor;
130 rclc_support_t support;
131 rcl_allocator_t allocator;
132 rcl_node_t node;
133
134 #define RCCHECK(fn) { \
135     rcl_ret_t temp_rc = fn; \
136     if ((temp_rc != RCL_RET_OK)) { \
137         error_loop(); \
138     } \
139 }
140
141 #define RCSOFTCHECK(fn) { \
142     rcl_ret_t temp_rc = fn; \
143     if ((temp_rc != RCL_RET_OK)) {} \
```

```
}

void error_loop()
{
    while (1)
    {
        delay(100);
    }
}

void subscription_callback(const void *msgin)
{
    const geometry_msgs__msg__Twist *cmd =
        (const geometry_msgs__msg__Twist *)msgin;

    float linear  = cmd->linear.x;
    float angular = cmd->angular.z;

    Serial.print("linear.x = ");
    Serial.print(linear);

    Serial.print(" | angular.z = ");
    Serial.println(angular);

    // Vorwaerts
    if (linear > 0.05 && abs(angular) < 0.05)
    {
        Serial.println("VORWAERTS");
        driveForward();
    }

    // Rueckwaerts
    else if (linear < -0.05 && abs(angular) < 0.05)
    {
        Serial.println("RUECKWAERTS");
        driveBackward();
    }

    // Links
    else if (angular > 0.05)
    {
        Serial.println("LINKS");
        turnLeft();
    }

    // Rechts
    else if (angular < -0.05)
    {
        Serial.println("RECHTS");
        turnRight();
    }

    else
    {
        Serial.println("STOP");

        // Stop-Befehl testen
        sendMotorCommand(
            LEFT_MOTOR_ID,
            MODE_CONTINUOUS,
            1,
            0
        );

        sendMotorCommand(
            RIGHT_MOTOR_ID,
            MODE_CONTINUOUS,
            0,
            0
        );
    }
}
```

```
217 void setup()
218 {
219     Serial.begin(115200);
220
221     MotorSerial.begin(
222         9600,
223         SERIAL_8N1,
224         -1,
225         MOTOR_TX_PIN
226     );
227
228     set_microros_wifi_transports(
229         ssid,
230         psk,
231         agent_ip,
232         agent_port
233     );
234
235     delay(2000);
236
237
238     allocator = rcl_get_default_allocator();
239
240     // ROS_DOMAIN_ID = 1
241     rcl_init_options_t init_options =
242         rcl_get_zero_initialized_init_options();
243
244     RCCHECK(
245         rcl_init_options_init(
246             &init_options,
247             allocator
248         )
249     );
250
251     RCCHECK(
252         rcl_init_options_set_domain_id(
253             &init_options,
254             1
255         )
256     );
257
258     RCCHECK(
259         rclc_support_init_with_options(
260             &support,
261             0,
262             NULL,
263             &init_options,
264             &allocator
265         )
266     );
267
268     RCCHECK(
269         rclc_node_init_default(
270             &node,
271             "micro_ros_robot",
272             "",
273             &support
274         )
275     );
276
277     RCCHECK(
278         rclc_subscription_init_default(
279             &subscriber,
280             &node,
281             ROSIDL_GET_MSG_TYPE_SUPPORT(
282                 geometry_msgs,
283                 msg,
284                 Twist
285             ),
286             "/cmd_vel"
287         )
288     );
289
```

```
290     RCCHECK(
291         rclc_executor_init(
292             &executor,
293             &support.context,
294             1,
295             &allocator
296         )
297     );
298 
299     RCCHECK(
300         rclc_executor_add_subscription(
301             &executor,
302             &subscriber,
303             &msg,
304             &subscription_callback,
305             ON_NEW_DATA
306         )
307     );
308 
309     Serial.println("Roboter bereit");
310 }
311 
312 void loop()
313 {
314     RCSOFTCHECK(
315         rclc_executor_spin_some(
316             &executor,
317             RCL_MS_TO_NS(20)
318         )
319     );
320 
321     delay(10);
322 }
```

Quellcode 16: C++ Code für den ESP32 zu Aufgabe 2.3

C 2.4 C++ Code

```
#include <Arduino.h>
#include <Wire.h>

#include <micro_ros_platformio.h>

#include <rcl/rcl.h>
#include <rcl/error_handling.h>
#include <rclc/rclc.h>
#include <rclc/executor.h>

#include <geometry_msgs/msg/twist.h>
#include <std_msgs/msg/float32.h>

#define WIFI_SSID       "WLAN-Name"
#define WIFI_PASSWORD   "Passwort"

IPAddress agent_ip(192, 168, 2, 218);
uint16_t agent_port = 8888;

#define MOTOR_TX_PIN 17
#define START_BYTE   0xAA

#define LEFT_MOTOR_ID   1
#define RIGHT_MOTOR_ID  2
#define MOTOR_PARAM 3

HardwareSerial MotorSerial(2);


#define TRIG_PIN 26
#define ECHO_PIN 25

#define SDA_PIN 21
#define SCL_PIN 22

#define MPU_ADDR       0x68
#define PWR_MGMT_1     0x6B
#define GYRO_CONFIG    0x1B
#define GYRO_ZOUT_H    0x47

float gyroBiasZ = 0.0;

rcl_node_t node;

rcl_subscription_t cmd_vel_subscriber;

rcl_publisher_t distance_publisher;
rcl_publisher_t gyro_publisher;

geometry_msgs__msg__Twist cmd_vel_msg;

std_msgs__msg__Float32 distance_msg;
std_msgs__msg__Float32 gyro_msg;

rclc_executor_t executor;

rcl_allocator_t allocator;
rclc_support_t support;

#define RCCHECK(fn) {                                   \
    rcl_ret_t temp_rc = fn;                             \
    if ((temp_rc != RCL_RET_OK))                        \
    {                                                   \
        Serial.print("micro-ROS Fehler: ");              \
        Serial.println(temp_rc);                        \
        errorLoop();                                    \
    }                                                   \
}

#define RCSOFTCHECK(fn) {                               \
```

```
    rcl_ret_t temp_rc = fn;                                 \
    if ((temp_rc != RCL_RET_OK))                            \
    {                                                       \
        Serial.print("micro-ROS Publish Fehler: ");         \
        Serial.println(temp_rc);                            \
    }                                                       \
}

void errorLoop()
{
    while (1)
    {
        delay(100);
    }
}

void sendMotorCommand(
    uint8_t id,
    uint8_t mode,
    uint8_t dir,
    uint16_t param)
{
    MotorSerial.write(START_BYTE);
    MotorSerial.write(id);
    MotorSerial.write(mode);
    MotorSerial.write(dir);

    MotorSerial.write((uint8_t)((param >> 8) & 0xFF));
    MotorSerial.write((uint8_t)(param & 0xFF));

    MotorSerial.flush();
}

void stopMotors()
{
    sendMotorCommand(LEFT_MOTOR_ID, 1, 1, 0);
    sendMotorCommand(RIGHT_MOTOR_ID, 1, 0, 0);
}

void moveForward()
{

    sendMotorCommand(
        LEFT_MOTOR_ID,
        1,
        1,
        MOTOR_PARAM);

    sendMotorCommand(
        RIGHT_MOTOR_ID,
        1,
        0,
        MOTOR_PARAM);
}

void moveBackward()
{

    sendMotorCommand(
        LEFT_MOTOR_ID,
        1,
        0,
        MOTOR_PARAM);

    sendMotorCommand(
        RIGHT_MOTOR_ID,
        1,
        1,
        MOTOR_PARAM);
}

void turnLeft()
{
```

```
144
145     sendMotorCommand(
146         LEFT_MOTOR_ID,
147         1,
148         0,
149         MOTOR_PARAM);
150
151     sendMotorCommand(
152         RIGHT_MOTOR_ID,
153         1,
154         0,
155         MOTOR_PARAM);
156 }
157
158 void turnRight()
159 {
160
161     sendMotorCommand(
162         LEFT_MOTOR_ID,
163         1,
164         1,
165         MOTOR_PARAM);
166
167     sendMotorCommand(
168         RIGHT_MOTOR_ID,
169         1,
170         1,
171         MOTOR_PARAM);
172 }
173
174 void cmdVelCallback(const void *msgin)
175 {
176     const geometry_msgs__msg__Twist *msg =
177         (const geometry_msgs__msg__Twist *)msgin;
178
179     float linear = msg->linear.x;
180     float angular = msg->angular.z;
181
182     Serial.print("cmd_vel: linear=");
183     Serial.print(linear);
184     Serial.print(" angular=");
185     Serial.println(angular);
186
187     // Vorwaerts
188     if (linear > 0.05 && fabs(angular) < 0.05)
189     {
190         moveForward();
191     }
192
193     // Rckwaerts
194     else if (linear < -0.05 && fabs(angular) < 0.05)
195     {
196         moveBackward();
197     }
198
199     // Links
200     else if (angular > 0.05)
201     {
202         turnLeft();
203     }
204
205     // Rechts
206     else if (angular < -0.05)
207     {
208         turnRight();
209     }
210
211     // Stop
212     else
213     {
214         stopMotors();
215     }
216 }
```

```
217
218 float measureDistance()
219 {
220     digitalWrite(TRIG_PIN, LOW);
221     delayMicroseconds(2);
222
223     digitalWrite(TRIG_PIN, HIGH);
224     delayMicroseconds(10);
225
226     digitalWrite(TRIG_PIN, LOW);
227
228     unsigned long duration =
229         pulseIn(ECHO_PIN, HIGH, 30000);
230
231     if (duration == 0)
232     {
233         return -1.0;
234     }
235
236     float distance_cm =
237         duration * 0.0343 / 2.0;
238
239     return distance_cm / 100.0;
240 }
241
242 void writeMPURegister(uint8_t reg, uint8_t value)
243 {
244     Wire.beginTransmission(MPU_ADDR);
245     Wire.write(reg);
246     Wire.write(value);
247     Wire.endTransmission();
248 }
249
250 int16_t readGyroZRaw()
251 {
252     Wire.beginTransmission(MPU_ADDR);
253     Wire.write(GYRO_ZOUT_H);
254
255     Wire.endTransmission(false);
256
257     Wire.requestFrom(
258         MPU_ADDR,
259         (uint8_t)2,
260         (uint8_t)true);
261
262     if (Wire.available() < 2)
263     {
264         return 0;
265     }
266
267     int16_t value =
268         ((int16_t)Wire.read() << 8) |
269         Wire.read();
270
271     return value;
272 }
273
274 float readGyroZ()
275 {
276
277     return readGyroZRaw() / 65.5;
278 }
279
280 void calibrateGyro()
281 {
282     const int samples = 200;
283
284     float sum = 0.0;
285
286     Serial.println();
287     Serial.println("Gyro-Kalibrierung");
288     Serial.println("Roboter NICHT bewegen!");
289
```

```
290     delay(1000);
291
292     for (int i = 0; i < samples; i++)
293     {
294         sum += readGyroZ();
295         delay(10);
296     }
297
298     gyroBiasZ = sum / samples;
299
300     Serial.print("Gyro-Bias Z = ");
301     Serial.print(gyroBiasZ, 3);
302     Serial.println(" deg/s");
303
304     Serial.println(
305         "Gyro-Kalibrierung abgeschlossen");
306 }
307
308 void setup()
309 {
310     Serial.begin(115200);
311
312     delay(1000);
313
314     Serial.println();
315     Serial.println("==============================");
316     Serial.println("ROS2 Turtlebot startet");
317     Serial.println("==============================");
318
319     MotorSerial.begin(
320         9600,
321         SERIAL_8N1,
322         -1,
323         MOTOR_TX_PIN);
324
325     stopMotors();
326
327     Serial.println("Motor UART gestartet");
328
329     pinMode(TRIG_PIN, OUTPUT);
330     pinMode(ECHO_PIN, INPUT);
331
332     digitalWrite(TRIG_PIN, LOW);
333
334     Serial.println("HC-SR04 gestartet");
335
336     Wire.begin(SDA_PIN, SCL_PIN);
337
338     writeMPURegister(PWR_MGMT_1, 0x00);
339
340     delay(100);
341
342     writeMPURegister(GYRO_CONFIG, 0x08);
343
344     calibrateGyro();
345
346     Serial.println();
347     Serial.println("Starte micro-ROS WiFi...");
348
349     set_microros_wifi_transports(
350         WIFI_SSID,
351         WIFI_PASSWORD,
352         agent_ip,
353         agent_port);
354
355     delay(2000);
356
357     allocator = rcl_get_default_allocator();
358
359
360     // ROS Domain ID = 1
361     rcl_init_options_t init_options =
362         rcl_get_zero_initialized_init_options();
```

```
363
364     RCCHECK(
365         rcl_init_options_init(
366             &init_options,
367             allocator));
368
369     RCCHECK(
370         rcl_init_options_set_domain_id(
371             &init_options,
372             1));
373
374     RCCHECK(
375         rclc_support_init_with_options(
376             &support,
377             0,
378             NULL,
379             &init_options,
380             &allocator));
381
382     RCCHECK(
383         rclc_node_init_default(
384             &node,
385             "esp32_robot",
386             "",
387             &support));
388
389     RCCHECK(
390         rclc_subscription_init_default(
391             &cmd_vel_subscriber,
392             &node,
393             ROSIDL_GET_MSG_TYPE_SUPPORT(
394                 geometry_msgs,
395                 msg,
396                 Twist),
397             "/cmd_vel"));
398
399     RCCHECK(
400         rclc_publisher_init_default(
401             &distance_publisher,
402             &node,
403             ROSIDL_GET_MSG_TYPE_SUPPORT(
404                 std_msgs,
405                 msg,
406                 Float32),
407             "/distance"));
408
409     RCCHECK(
410         rclc_publisher_init_default(
411             &gyro_publisher,
412             &node,
413             ROSIDL_GET_MSG_TYPE_SUPPORT(
414                 std_msgs,
415                 msg,
416                 Float32),
417             "/gyro_z"));
418
419     RCCHECK(
420         rclc_executor_init(
421             &executor,
422             &support.context,
423             1,
424             &allocator));
425
426
427     RCCHECK(
428         rclc_executor_add_subscription(
429             &executor,
430             &cmd_vel_subscriber,
431             &cmd_vel_msg,
432             &cmdVelCallback,
433             ON_NEW_DATA));
434
435
```

```
436     Serial.println();
437     Serial.println("==============================");
438     Serial.println("micro-ROS bereit");
439     Serial.println("Topics:");
440     Serial.println("  SUB: /cmd_vel");
441     Serial.println("  PUB: /distance");
442     Serial.println("  PUB: /gyro_z");
443     Serial.println("==============================");
444 }
445 
446 void loop()
447 {
448     RCSOFTCHECK(
449         rclc_executor_spin_some(
450             &executor,
451             RCL_MS_TO_NS(10)));
452 
453     static unsigned long lastSensorTime = 0;
454 
455     unsigned long now = millis();
456 
457     if (now - lastSensorTime >= 100)
458     {
459         lastSensorTime = now;
460 
461         float distance = measureDistance();
462 
463         distance_msg.data = distance;
464 
465         RCSOFTCHECK(
466             rcl_publish(
467                 &distance_publisher,
468                 &distance_msg,
469                 NULL));
470 
471         float gyroZ =
472             readGyroZ() - gyroBiasZ;
473 
474         if (fabs(gyroZ) < 1.0)
475         {
476             gyroZ = 0.0;
477         }
478 
479         gyro_msg.data = gyroZ;
480 
481         RCSOFTCHECK(
482             rcl_publish(
483                 &gyro_publisher,
484                 &gyro_msg,
485                 NULL));
486 
487         Serial.print("Distance: ");
488 
489         if (distance < 0)
490         {
491             Serial.print("kein Echo");
492         }
493         else
494         {
495             Serial.print(distance, 3);
496             Serial.print(" m");
497         }
498 
499         Serial.print(" | Gyro Z: ");
500         Serial.print(gyroZ, 2);
501         Serial.println(" deg/s");
502     }
503 }
```

Quellcode 17: C++ Code für den ESP32 zu Aufgabe 2.4

D obstacle_avoidance.py

/home/vboxuser/ros2_ws/src/turtle_controller/turtle_controller/obstacle_avoidance.py

```python
import rclpy
from rclpy.node import Node

from geometry_msgs.msg import Twist
from std_msgs.msg import Float32


class ObstacleAvoidance(Node):

    def __init__(self):
        super().__init__('obstacle_avoidance')

        self.cmd_pub = self.create_publisher(
            Twist,
            '/cmd_vel',
            10
        )

        self.distance_sub = self.create_subscription(
            Float32,
            '/distance',
            self.distance_callback,
            10
        )

        self.gyro_sub = self.create_subscription(
            Float32,
            '/gyro_z',
            self.gyro_callback,
            10
        )

        self.timer = self.create_timer(
            0.05,
            self.control_loop
        )

        self.DISTANCE_THRESHOLD = 0.25  # Meter
        self.BACKWARD_TIME = 1.0  # Sekunden
        self.STOP_TIME = 0.3  # Sekunden
        self.TARGET_ANGLE = 90.0  # Grad

        self.LINEAR_FORWARD = 1.0
        self.LINEAR_BACKWARD = -1.0
        self.ANGULAR_LEFT = 1.0


        self.state = 'FORWARD'

        self.distance = -1.0
        self.gyro_z = 0.0

        self.angle = 0.0

        self.state_start_time = (
            self.get_clock().now().nanoseconds / 1e9
        )

        self.last_gyro_time = None

        self.get_logger().info(
            'Obstacle Avoidance gestartet'
        )

    def distance_callback(self, msg):

        self.distance = msg.data

```

```
69
70      def gyro_callback(self, msg):
71
72          self.gyro_z = msg.data
73
74          if self.state != 'TURN':
75              self.last_gyro_time = None
76              return
77
78          now = self.get_clock().now().nanoseconds / 1e9
79
80          if self.last_gyro_time is None:
81              self.last_gyro_time = now
82              return
83
84          dt = now - self.last_gyro_time
85          self.last_gyro_time = now
86
87          self.angle += self.gyro_z * dt
88
89      def publish_stop(self):
90
91          msg = Twist()
92
93          msg.linear.x = 0.0
94          msg.angular.z = 0.0
95
96          self.cmd_pub.publish(msg)
97
98      def publish_forward(self):
99
100         msg = Twist()
101
102         msg.linear.x = self.LINEAR_FORWARD
103         msg.angular.z = 0.0
104
105         self.cmd_pub.publish(msg)
106
107     def publish_backward(self):
108
109         msg = Twist()
110
111         msg.linear.x = self.LINEAR_BACKWARD
112         msg.angular.z = 0.0
113
114         self.cmd_pub.publish(msg)
115
116     def publish_turn_left(self):
117
118         msg = Twist()
119
120         msg.linear.x = 0.0
121         msg.angular.z = self.ANGULAR_LEFT
122
123         self.cmd_pub.publish(msg)
124
125     def change_state(self, new_state):
126
127         self.state = new_state
128
129         self.state_start_time = (
130             self.get_clock().now().nanoseconds / 1e9
131         )
132
133         self.get_logger().info(
134             f'Neuer Zustand: {new_state}'
135         )
136
137     def control_loop(self):
138
139         now = self.get_clock().now().nanoseconds / 1e9
140
141         elapsed = now - self.state_start_time
```

```
142
143         if self.state == 'FORWARD':
144
145             self.publish_forward()
146
147             if (
148                 self.distance > 0.0
149                 and self.distance < self.DISTANCE_THRESHOLD
150             ):
151
152                 self.get_logger().info(
153                     f'Hindernis erkannt: '
154                     f'{self.distance:.3f} m'
155                 )
156
157                 self.change_state('BACKWARD')
158
159         elif self.state == 'BACKWARD':
160
161             self.publish_backward()
162
163             if elapsed >= self.BACKWARD_TIME:
164
165                 self.publish_stop()
166
167                 self.change_state('STOP_BEFORE_TURN')
168
169         elif self.state == 'STOP_BEFORE_TURN':
170
171             self.publish_stop()
172
173             if elapsed >= self.STOP_TIME:
174
175                 self.angle = 0.0
176                 self.last_gyro_time = None
177
178                 self.get_logger().info(
179                     '90-Grad-Drehung gestartet'
180                 )
181
182                 self.change_state('TURN')
183
184         elif self.state == 'TURN':
185
186             self.publish_turn_left()
187
188             self.get_logger().info(
189                 f'Winkel: {self.angle:.1f} deg',
190                 throttle_duration_sec=0.5
191             )
192
193             if abs(self.angle) >= self.TARGET_ANGLE:
194
195                 self.publish_stop()
196
197                 self.get_logger().info(
198                     f'Drehung beendet: '
199                     f'{self.angle:.1f} Grad'
200                 )
201
202                 self.change_state('STOP_AFTER_TURN')
203
204         elif self.state == 'STOP_AFTER_TURN':
205
206             self.publish_stop()
207
208             if elapsed >= self.STOP_TIME:
209
210                 self.change_state('FORWARD')
211
212
213 def main(args=None):
214
```

```
215     rclpy.init(args=args)
216
217     node = ObstacleAvoidance()
218
219     try:
220         rclpy.spin(node)
221
222     except KeyboardInterrupt:
223         pass
224
225     node.publish_stop()
226
227     node.destroy_node()
228
229     rclpy.shutdown()
230
231
232 if __name__ == '__main__':
233     main()
```

Quellcode 18: Implementierung der Obstacle Avoidance Node

E wall_follower.py

```python
import math

import rclpy
from rclpy.node import Node

from sensor_msgs.msg import LaserScan
from geometry_msgs.msg import Twist


class WallFollower(Node):

    def __init__(self):
        super().__init__('wall_follower')

        # Zielabstand zur Wand in Metern
        self.target_distance = 0.40

        # Sicherheitsabstand zur Wand.
        self.min_distance = 0.30

        # Geschwindigkeit
        self.forward_speed = 0.15       # m/s
        self.turn_speed = 0.8           # rad/s

        self.kp = 1.5

        # Maximale Lenkgeschwindigkeit
        self.max_angular_speed = 0.8

        # Abstand, ab dem ein Hindernis vorne erkannt wird
        self.front_obstacle_distance = 0.45

        self.scan_subscriber = self.create_subscription(
            LaserScan,
            '/scan',
            self.scan_callback,
            10
        )

        self.cmd_publisher = self.create_publisher(
            Twist,
            '/cmd_vel',
            10
        )

        self.timer = self.create_timer(
            0.1,
            self.control_loop
        )

        self.latest_scan = None

        self.get_logger().info('Wall Follower gestartet.')
        self.get_logger().info(
            f'Zielabstand: {self.target_distance:.2f} m'
        )

    def scan_callback(self, msg):
        self.latest_scan = msg

    def get_distance(self, scan, angle_deg):

        angle_rad = math.radians(angle_deg)

        #  5  Grad um den gewuenschten Winkel
        angle_min = angle_rad - math.radians(5)
        angle_max = angle_rad + math.radians(5)

        index_min = int(
            (angle_min - scan.angle_min) / scan.angle_increment
```

```
        )

        index_max = int(
            (angle_max - scan.angle_min) / scan.angle_increment
        )

        index_min = max(0, index_min)
        index_max = min(len(scan.ranges) - 1, index_max)

        distances = []

        for i in range(index_min, index_max + 1):

            distance = scan.ranges[i]

            if math.isfinite(distance):
                if scan.range_min <= distance <= scan.range_max:
                    distances.append(distance)

        if not distances:
            return float('inf')

        return min(distances)

    def control_loop(self):

        # Noch keine Laserdaten erhalten
        if self.latest_scan is None:
            self.stop_robot()
            return

        scan = self.latest_scan

        # Rechts
        right_distance = self.get_distance(scan, -90)

        # Vorne rechts
        front_right_distance = self.get_distance(scan, -45)

        # Vorne
        front_distance = self.get_distance(scan, 0)

        cmd = Twist()

        if front_distance < self.front_obstacle_distance:

            self.get_logger().info(
                f'Hindernis vorne: {front_distance:.2f} m - '
                'drehe links'
            )

            cmd.linear.x = 0.0
            cmd.angular.z = self.turn_speed

            self.cmd_publisher.publish(cmd)
            return

        if right_distance < self.min_distance:

            error = self.target_distance - right_distance

            angular = self.kp * error

            angular = max(
                -self.max_angular_speed,
                min(self.max_angular_speed, angular)
            )

            cmd.linear.x = self.forward_speed * 0.6
            cmd.angular.z = angular

        else:

```

```
144             # Fehler:
145             #
146             # positive Zahl = Wand zu weit weg
147             # negative Zahl = Wand zu nah
148 
149             error = self.target_distance - right_distance
150 
151             # Bei einer rechten Wand:
152             #
153             # Wand zu weit weg  -> nach rechts drehen
154             # Wand zu nah        -> nach links drehen
155 
156             angular = -self.kp * error
157 
158             angular = max(
159                 -self.max_angular_speed,
160                 min(self.max_angular_speed, angular)
161             )
162 
163             cmd.linear.x = self.forward_speed
164             cmd.angular.z = angular
165 
166         self.cmd_publisher.publish(cmd)
167 
168     def stop_robot(self):
169 
170         cmd = Twist()
171 
172         cmd.linear.x = 0.0
173         cmd.linear.y = 0.0
174         cmd.linear.z = 0.0
175 
176         cmd.angular.x = 0.0
177         cmd.angular.y = 0.0
178         cmd.angular.z = 0.0
179 
180         self.cmd_publisher.publish(cmd)
181 
182 
183 def main(args=None):
184 
185     rclpy.init(args=args)
186 
187     node = WallFollower()
188 
189     try:
190         rclpy.spin(node)
191 
192     except KeyboardInterrupt:
193         pass
194 
195     finally:
196         node.stop_robot()
197         node.destroy_node()
198         rclpy.shutdown()
199 
200 
201 if __name__ == '__main__':
202     main()
```

Quellcode 19: Implementierung der Wall Follower Node für den Turtlebot